OpenURMA: A Clean-Room Open Implementation of the Unified Bus Protocol

Bojie Li

Modern datacenter RDMA is bottlenecked at the network interface, not at the wire. At the scale of AI-training collectives and HPC all-to-alls, a NIC running RoCE or InfiniBand must hold per-connection state for every (application, remote-endpoint) pair — hundreds of megabytes of context at 1024-application fanout — and pays a four-traversal PCIe round trip on the smallest 64-byte operation, inflating end-to-end latency by an order of magnitude beyond the wire delay. Both costs are structural consequences of the Queue-Pair-over-PCIe abstraction that commodity RDMA inherits from InfiniBand.

Huawei's *Unified Bus* (UB) is a public 2025 specification that removes both costs by changing the abstraction. Its transport protocol decouples per-application endpoint state from per-host reliable-transport state, so connection context grows additively rather than multiplicatively; it exposes ordering as an opt-in surface so applications pay gating only when they need it; and it reaches remote memory through a CPU's native load/store instructions to a controller on the on-chip bus rather than through verbs to a PCIe-attached device. The protocol ships in Huawei's Ascend 950 silicon, which is closed.

OpenURMA is the first clean-room open implementation of the Unified Bus protocol's transport and transaction layers, written from the public specification. We realise it at three modeling tiers — synthesisable RTL on Alveo U50, a cycle-level two-node SystemC simulator that accounts for both endpoints of the wire, and a gem5 full-system scaffold in which two ARM CPUs run real user binaries against the SystemC NIC — with a matched OpenRoCE (RoCEv2 RC) baseline at every tier. The contribution is not the architecture, which is Huawei's; it is the implementation, the multi-tier evaluation harness, and the controlled comparison that closed silicon does not admit.

The artifact lets the community measure the protocol's costs cell-by-cell, locate its operating points against RoCE on identical infrastructure, and prototype follow-on transports on a common substrate. We use it to show that UB's three architectural choices are mutually load-bearing: removing any one and keeping the others on a PCIe-attached NIC reproduces RoCE's costs. On the canonical 64-byte cache-line fetch from remote memory — executed as a LOAD on UB §8.3 and as a READ on RoCEv2 RC — the UB load/store path delivers ≈500 ns end-to-end CPU-to-remote-DRAM-to-CPU latency, **4.37**× below the matched RoCEv2 baseline (2186 ns), and sustains 2.80× higher steady-state work-request throughput on the verb-driven path. The whole design fits in roughly 14% of a U50's LUT budget.

1. Introduction

A modern datacenter network is two interconnects fused into one. From the application's view, RDMA looks like a load or store to remote memory — bus-like, low-latency, address-and-go. From the NIC's view, every operation is a queued work request shuttled across PCIe to a peripheral device — network-like, with all of the network's per-message machinery and the peripheral's PCIe crossing. For two decades the fusion was tractable: connection counts stayed small, operation sizes stayed large, and the gap between the bus-like view and the network-like implementation could be papered over with cleverer software above the device. AI-training workloads broke both assumptions. A single job now exchanges 64-byte gradient updates across thousands of GPUs; the bus-like view's promises collide with the network-like implementation's costs, and the collision is structural.

The two costs of the collision are both consequences of one design stance: *the NIC is a peripheral*. Peripherals communicate with the CPU through a paired-queue programming model, so the NIC must hold per-pair connection state; peripherals sit behind PCIe, so every operation crosses the PCIe boundary. Under RoCEv2 RC, each (local application, remote endpoint) pair is a Queue Pair (QP) carrying ∼512 B of NIC state; at 1024 of each, the NIC's working set is ∼537 MB, past any commodity NIC's on-chip SRAM and into the regime where every operation pays a PCIe round trip to refetch its QP [23, 18, 20, 46]. The same operation pays a second cost even when state stays on chip: a 64-byte READ spends roughly 1.5 μs of its ∼2 μs round trip on four PCIe traversals (doorbell MMIO, work-queue DMA fetch, completion DMA write, CPU poll-miss across PCIe coherence), while the wire delivers in ∼200 ns. Software cannot move the peripheral; hardware inside the peripheral cannot move PCIe. What removes both costs is moving the peripheral.

Huawei's *Unified Bus* (UB) [15] is a 2025 specification that moves the peripheral. The NIC is no longer behind PCIe but on the on-chip bus, addressed by the CPU through ordinary load and store instructions; connection state is no longer per-application-pair but per-host-pair; ordering is no longer always-on but opt-in. Huawei's Ascend 950 NPU [16] ships UB at commodity scale, but the silicon is closed and the spec is unaccompanied by a public implementation that researchers can measure, instrument, or prototype against.

The three architectural moves UB makes are not independent choices; they form a chain in which each move makes the next possible (Figure 1).

1. **Split the transport layer.** The Queue Pair fused application identity with transport reliability into one

object, so state had to scale with the product of the two counts. UB separates the two: per-application endpoint state lives on a *Jetty*; per-remote-host transport state lives on a *TP Channel*. Per-NIC state grows additively, as $O(N+M)$ instead of $O(N \cdot M)$. Bounding state is what makes (2) possible.

2. **Move the controller onto the on-chip bus.** A NIC whose working set fits in on-chip SRAM can live next to the CPU on the on-chip bus rather than behind PCIe; a NIC whose working set spills to host DRAM cannot, because each spill becomes a host-memory access. Putting the controller on-bus is what makes (3) possible.
3. **Admit a load/store data path.** Once the controller is on the on-chip bus, a CPU's load or store instruction can reach it directly. The four PCIe traversals collapse into a single on-chip-bus crossing, and the work-queue and completion-queue machinery elides for small synchronous operations.

A fourth move — opt-in ordering — rides on the same per-application counters the first move provisions, so it costs zero extra pipeline cycles on operations that do not request gating, while letting workloads that need a barrier pay only for that barrier.

OpenURMA is a clean-room open implementation of the Unified Bus protocol's transport and transaction layers, written from the public specification. The same artifact is observable at three modeling tiers, each load-bearing for a different sub-claim: **synthesisable RTL on Alveo U50** carries the state and area claims; a **cycle-level two-node SystemC simulator** carries per-operation latency and sustained throughput across 388 sweep configurations and all six verb categories with both endpoints modeled; a **gem5 full-system scaffold** carries the CPU-to-remote-DRAM end-to-end claim by running real ARM user binaries against the SystemC NIC. A parallel *OpenRoCE* stack (RoCEv2 RC) on the same toolchain and harness anchors an apples-to-apples baseline at every tier, so both stacks' numbers come from identical modeling assumptions rather than from published vendor data. On a 64-byte READ in the two-node simulator, the load/store path delivers ≈500 ns end-to-end on the LOAD verb — **4.37**× below the RoCEv2 READ baseline (2186 ns) for the same CPU-fetches-64 B operation.

Contributions.

- **First clean-room open implementation of the Unified Bus protocol.** Synthesisable RTL for the transport and transaction layers, closing 322 MHz post-route at roughly 14% of an Alveo U50's LUT budget.
- **Apples-to-apples OpenRoCE baseline.** A RoCEv2 RC implementation on the same toolchain, target, and harness.
- **Multi-tier evaluation infrastructure.** A cycle-level two-node SystemC simulator that accounts for both endpoints of the wire, plus a gem5 full-system scaffold that puts a CPU in the loop; jointly necessary to measure the end-to-end CPU-to-remote-DRAM path UB claims to shorten.
- **Ten cross-cutting validation experiments,** including a model-validation step that reproduces published ConnectX-7 RDMA WRITE latency within ±5%, congestion-control dynamics against DCQCN, and a YCSB-A port.

OpenURMA is released at `https://github.com/bojieli/OpenURMA`.

Roadmap. §2 places UB in context against the peripheral-NIC abstraction and three decades of patches around it. §3 walks the implementation's design choices. §4 reports the pipeline decomposition and timing-closure sweep. §5 sets up the evaluation harness. §§6–8 report microbenchmarks, the two-node end-to-end study, and ten cross-cutting validation experiments. §9 gives area and timing. §10 discusses limitations; §11 surveys related work.

2. Background

This section grounds the introduction's thesis in concrete mechanics: what makes the NIC a peripheral, why the peripheral stance imposes both costs by construction, why three decades of fixes around it leave the abstraction in place, and what Unified Bus replaces it with.

2.1 The peripheral-NIC abstraction

Every commodity RDMA NIC — InfiniBand HCAs, Mellanox/NVIDIA ConnectX, Broadcom Thor, Intel IPU — is a PCIe peripheral. The CPU sees it through a memory-mapped BAR; the NIC sees the CPU through DMA. The pair communicates by a Queue-Pair protocol: the CPU posts work-queue entries to host memory, signals via MMIO doorbells, and the NIC consumes them by DMA and writes completions back the same way. This programming model is itself a small network protocol spoken across PCIe, and the model fixes two properties of the system before the wire is even reached.

The QP fuses what should be two layers. A Queue Pair is named by a (local-application, remote-endpoint) pair and carries both transaction-layer state (work queues, permissions, the identity of *who is calling*) and transport-layer state (sequence numbers, retransmit window, RTO timer, congestion state — the bookkeeping of *how packets are delivered*). Textbook protocol design separates these layers; the QP collapses them into one object whose unit of accounting is the application pair, not the application count or the peer count. Total NIC state therefore grows as $O(N \cdot M)$. At $(N, M)=(1024, 1024)$, $\sim$1 M QPs at $\sim$512 B each is $\sim$537 MB, beyond any commodity NIC's on-chip SRAM. When the working set spills to host DRAM, every operation pays a PCIe round trip to refetch its QP before it can even consult its sequence number [23, 46].

The PCIe attachment imposes four traversals per operation. Because the NIC is a peripheral, every oper-

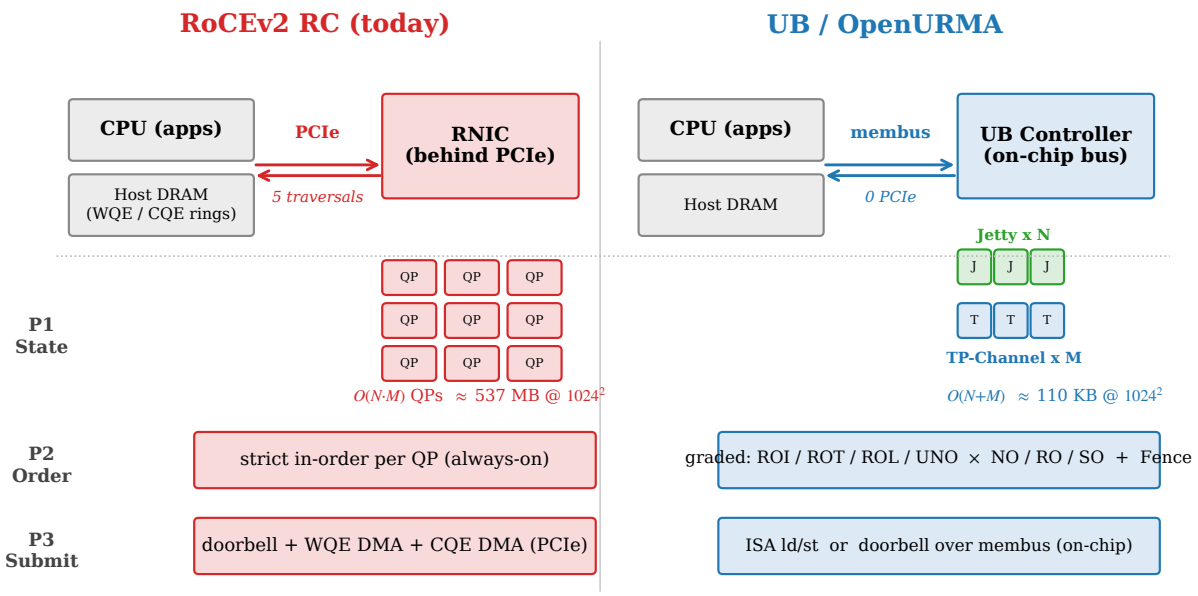


Figure 1: The three architectural moves and their dependencies. RoCEv2 RC (top) puts the NIC behind PCIe, holds one Queue Pair per (application, remote-endpoint) pair, and enforces strict order on every operation. UB (bottom) splits state along the transaction/transport line so it grows as $O(N+M)$; places the controller on the on-chip bus, where the CPU can reach it with loads and stores; and exposes ordering as four opt-in axes that reuse counters the layer split already provisions. The arrows mark the chain: bounded state is what lets the controller live on-bus; the on-bus controller is what lets the load/store path exist; the layer split is what makes ordering cheap.

ation crosses the PCIe boundary four times. On a 64-byte READ: the CPU writes a work-queue entry to host DRAM and then a doorbell to the NIC's MMIO BAR (traversal 1); the NIC DMA-reads the work-queue entry (traversal 2); the NIC emits the wire request, receives the response, and DMA-writes a completion entry to host DRAM (traversal 3); the CPU polls the completion queue and incurs a PCIe-coherence miss to fetch the freshly written line (traversal 4). Of the $\sim$2 μs round-trip time, $\sim$1.5 μs is consumed by these four traversals; the wire itself delivers in $\sim$200 ns. The traversals are not an implementation accident; they are what the model requires when the two communicating parties live in disjoint address spaces.

Both costs follow from one stance. Fixing either in isolation leaves the other in place: connection sharing (XRC, SRQ) reduces state but keeps the four PCIe traversals; doorbell coalescing and inline-WQE shortcuts trim one traversal but keep the QP fusion. The abstraction is what limits the optimisation budget.

2.2 Why prior fixes are patches

Three decades of work on RDMA scale fall in three buckets, none of which removes both costs. *Software above the device* — FaSST [18], 1RMA [41], Pony Express, the broader SmartNIC-pacing literature — works above the verb interface; it can change the application's view but not the peripheral attachment or the programming model. *Hardware inside the device* — SRNIC [46], StaR [45], the IRN loss-recovery line [35, 33, 13, 31], MP-RDMA [34], ConWeave [43] — rebuilds the NIC's internals symptom by symptom, but the NIC remains a PCIe peripheral with a Queue-Pair programming model. *Programmable substrates* — Tonic [3], NanoTransport [17], StRoM [40] — host new transports on FPGA or P4 fabric, but they are substrates, not new transports.

2.3 Unified Bus: the spec, the silicon, the gap

UB is the first RDMA-class specification since RoCEv2 (2014) to replace the abstraction rather than patch it. The 2025 spec spans physical through verb layers; this paper engages only transport and transaction, where the structural choices live. The protocol ships in Huawei's Ascend 950 NPU [16] with both the work-queue and load/store data paths as on-die blocks alongside the AI accelerators. The user-space verb library and kernel driver are open-source; the protocol implementation is closed. OpenURMA fills that gap with a clean-room implementation built from the public specification.

2.4 Making the NIC a peer of the CPU

UB stops treating the NIC as a peripheral the CPU programs and starts treating it as a peer the CPU shares an on-chip bus with (Figure 2). The spec-level mechanisms unfold in the order in which each enables the next: a layer split bounds state, bounding state lets the controller live on-bus, an on-bus controller admits a load/store data path. A fourth mechanism — opt-in ordering — rides on the layer split's per-application counters.

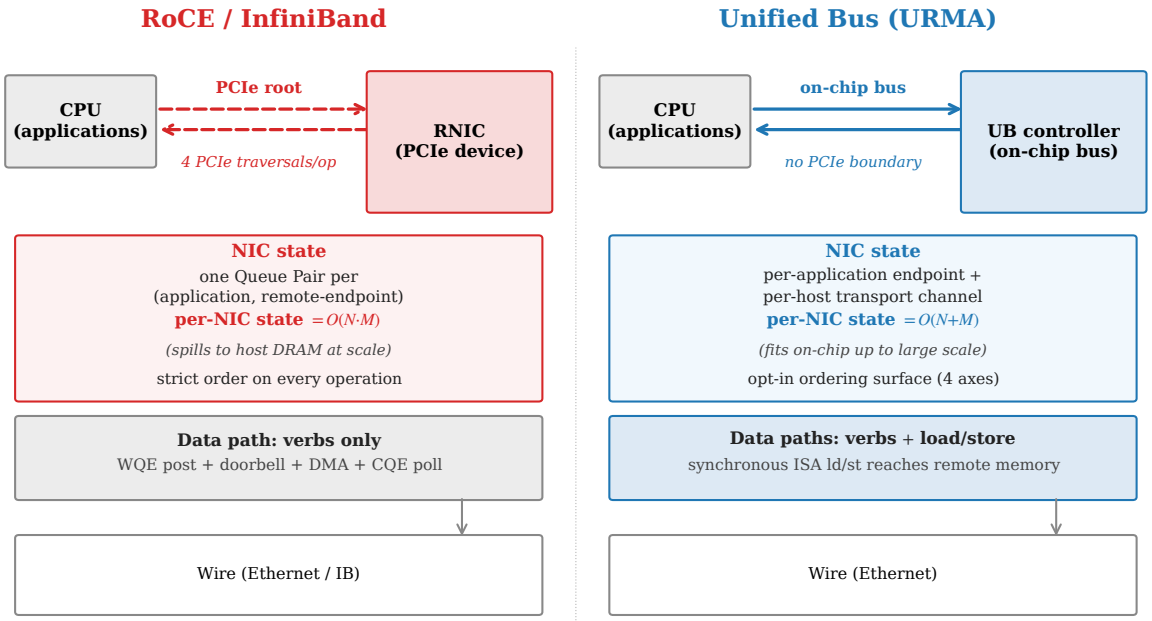


Figure 2: Architectural comparison. RoCE puts the NIC behind PCIe; it holds one Queue Pair per (application, remote-endpoint) pair, enforces strict order on every operation, and admits only the verb-driven data path. Unified Bus puts the controller on the on-chip bus; it holds per-application endpoint state separately from per-remote-host transport state, exposes ordering as an opt-in surface, and admits CPU load/store as a synchronous data path alongside the verb path.

Move 1: split transaction from transport. The QP fused two layers into one object, so state had to scale with their product. UB separates them: a *Jetty* holds the transaction-layer state for one local application (work queues, permissions, completion handle, ∼80 B); a *TP Channel* holds the transport-layer state for one remote host (sequence numbers, retransmit window, RTO timer, congestion state, ∼30 B) (Figure 3). Any Jetty addresses any remote endpoint through the shared per-host TP Channel pool, with the destination carried in the packet header rather than baked into a per-pair binding. State grows as $O(N{+}M)$: at $(N, M){=}(1024, 1024)$, ∼110 KB instead of 537 MB.

The split pays for itself with one extra lookup per outbound packet (which TP Channel by destination host) and a protocol surface that exposes Jetty and TP Channel as separate first-class objects with separate lifecycles. The QP's one-object-per-peer simplicity is the cost of admission.

Move 2: put the controller on the on-chip bus. At $O(N{+}M)$ state, the connection working set fits in on-chip SRAM at scales that previously forced spill to host DRAM. A NIC whose context lives on-chip can live next to the CPU on the on-chip bus rather than behind PCIe: the CPU sees the controller through ordinary memory-mapped regions, the controller sees the CPU's loads and stores at on-chip-bus latency, and the PCIe boundary collapses into a single bus crossing. The cost: the controller must sit on the same bus segment as the CPU it serves.

Move 3: admit a load/store data path. With the controller on-bus, a CPU load or store can reach an address aperture it owns (Figure 4). The controller turns each instruction into one wire transaction — no work-queue entry, no doorbell, no DMA, no separate completion — and the load blocks until the response returns. The four PCIe traversals collapse into one on-chip-bus crossing. The cost: only short synchronous operations (loads, stores, same-path atomics) are admissible; bulk and asynchronous traffic stays on the work-queue path. The two paths complement rather than compete: load/store wins where the operation is short and the latency budget is tight; the work-queue path wins where the operation is long and the throughput budget dominates.

Move 4: make ordering opt-in. This move sits outside the chain but on top of Move 1. RoCE RC enforces strict in-order delivery on every operation; UB exposes ordering as four orthogonal axes the application opts into per channel and per operation. The *service mode* controls how aggressively packets may reorder on the wire (relaxed admits multi-path spreading; strict forces single-path delivery); the *execution order* controls whether the issue stage may emit before the previous operation commits; the *fence* is an explicit application barrier; the *completion order* bit controls whether completions retire in arrival or issue order. A pure-fanout broadcast bypasses every gate; a barrier-after-collective pays exactly the gating its workload needs.

The gating indexes per-Jetty counters Move 1 already provisions, so opt-in ordering costs zero added pipeline cycles on operations that bypass gating. A second argu-

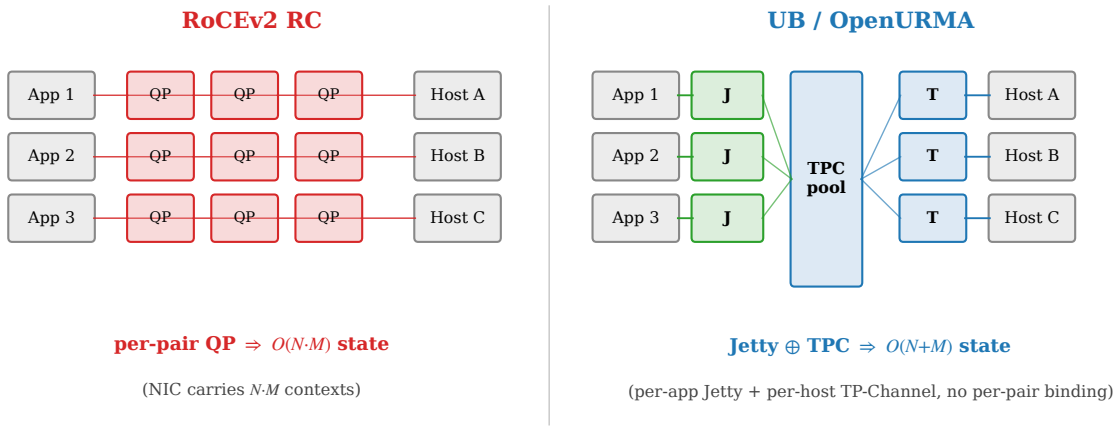


Figure 3: State models compared. RoCE binds one Queue Pair to every (application, remote-host) pair, so per-NIC state grows as $O(N{\cdot}M)$. UB decouples per-application state (Jetty) from per-host transport state (TP Channel); any Jetty addresses any remote endpoint through the shared TP Channel pool, and per-NIC state grows as $O(N{+}M)$.

ment: strict total order is not just expensive on the fast path but fragile under failure — a NIC promised in-order delivery cannot retire any completion when any earlier operation has stalled, so one slow path becomes a head-of-line block for every application on the channel. The opt-in surface lets each workload request the gating it needs without imposing it on the others.

2.5 The OpenClickNP toolchain

OpenURMA is built on OpenClickNP [25], a clean-room re-implementation of the ClickNP element model [27]. The model is a familiar one: a NIC pipeline is a directed graph of *elements*, each with typed input and output ports, local state, and a per-cycle handler. The toolchain lowers a single source description through three back-ends: a software emulator for unit testing, a cycle-accurate SystemC simulator for performance measurement, and a Vitis HLS back-end that emits synthesisable C++ for Vivado synthesis on the Alveo U50. The same source produces all three artifacts. The relevance to this paper is twofold: the model makes element-level cost (LUTs, pipeline-II, BRAM) directly measurable, and the three back-ends are how OpenURMA produces both the RTL tier and the SystemC tier from one description.

3. Design

This section describes how OpenURMA realises Unified Bus in RTL. We organise the description around the three architectural commitments introduced in §2.4 — bounded state, opt-in ordering, and the load/store data path — followed by the composition that makes the three mutually load-bearing. The element-by-element decomposition, port wirings, and HLS-pragma specifics are deferred to §4; this section keeps the discussion at the level of how the pipeline embodies each architectural choice and what trade-offs that embodiment forces.

3.1 Realising bounded state

The Jetty / TP Channel split maps onto two pipeline regions. The transmit path consults a per-Jetty table to admit and gate the work request, then hands it to a per-host TP Channel scheduler that attaches the sequence number, picks the multi-path lane, and queues the packet. The receive path inverts the flow: per-TP-Channel logic validates and reorders by sequence number, then a per-Jetty table maps the destination header to the right receive queue. Two tables, two indices, two scaling regimes. The pipeline pays one extra header field per packet (the destination Jetty id) and one extra lookup per side of the wire; in exchange, nothing in the pipeline binds a transmit context to a receive context, so the protocol's $O(N{+}M)$ scaling becomes the pipeline's. At $(N, M){=}(1024, 1024)$ the two-table working set is ∼110 KB versus ∼537 MB for the matched RoCE QP table (§9.3).

3.2 Realising opt-in ordering

The four ordering axes become four gating stages on the pipeline. Each decides at one well-defined point whether the operation in flight may proceed; together they form a mesh that adds zero pipeline cycles to operations whose mode bypasses every gate. The trick is in what the gating logic indexes against.

Reused counters. Each gate reads the same three inputs: a per-Jetty sequence counter ("do prior operations from this application still need to complete?"), a per-channel outstanding-window counter ("can the wire absorb another?"), and a per-Jetty fence latch. The state model in §3.1 already holds all three — the spec defines the relevant sequence numbers as per-Jetty objects, and the per-Jetty table carries them regardless. The ordering surface reads from counters the state model writes to; it does not add a parallel bookkeeping structure.

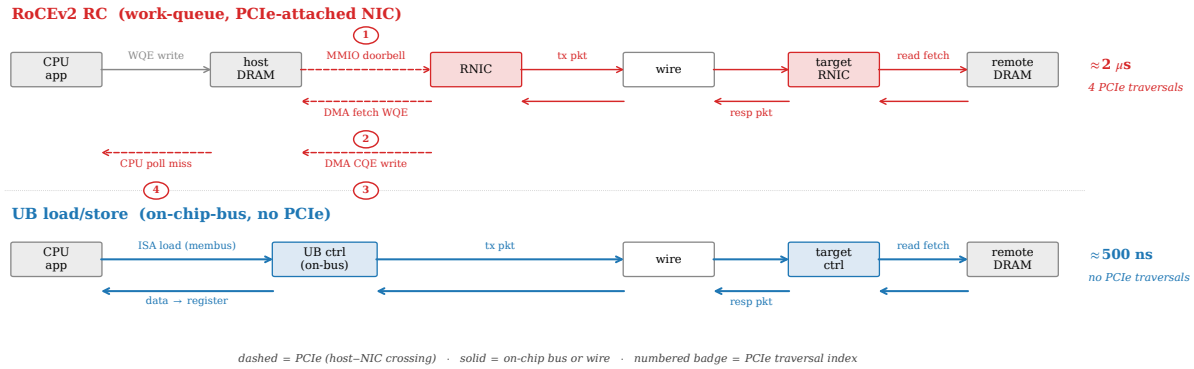


Figure 4: Per-operation data path for a small synchronous read. The traditional work-queue-driven path (top) traverses four PCIe crossings — doorbell over MMIO, work-queue-entry DMA fetch, completion-entry DMA write, and a CPU poll-miss across PCIe coherence. The Unified Bus load/store path (bottom) replaces all four with on-chip-bus crossings: a CPU load instruction reaches the controller, the controller emits the wire packet, the response arrives, and the data lands directly in the CPU register that issued the load.

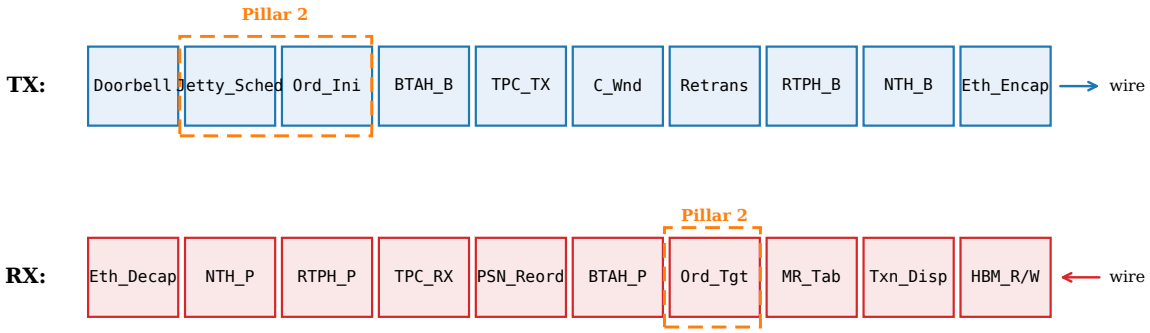


Figure 5: OpenURMA's NIC pipeline, organised by protocol layer. The transaction layer (top half) handles application-level concerns — Jetty scheduling, ordering, memory-region checks, atomic operations, completion generation. The transport layer (bottom half) handles per-host concerns — packet sequence numbers, retransmission, congestion control, multi-path. The two layers communicate through flit-typed boundaries; the load/store data path (§3.3) bypasses the transport layer entirely.

Two reorder buffers, not one. A subtler choice is to carry *two* reorder buffers serving disjoint contracts (Figure 6). One acts at the transport layer on packet sequence numbers and delivers byte-correct flits regardless of any application's ordering choice, so a TP Channel can multiplex multiple applications without one application's wire-byte gaps stalling another's. The other acts at the transaction layer on per-Jetty sequence numbers and delivers in-issue-order completions regardless of wire-byte order, so the transmit path can spread packets across multiple network paths without breaking the application's view. Collapsing the two would couple the contracts: every multi-path packet gap would block the completion stream; every per-application dependency stall would back-pressure the wire. The separation is what makes unordered service mode and strict mode correctness-safe at the same time. §11 contrasts this with UEC's and MP-RDMA's per-transaction bitmap approach.

3.3 Realising the load/store path

The work-queue-driven path runs ten pipeline stages cold (work-queue scheduling, ordering, header build, transport accounting, congestion check, multi-path lane select, retransmit-buffer write, framing, transmit). For a small synchronous operation, every one is overhead the abstraction does not need.

The load/store path drops them. A CPU `ld` or `st` to the controller's address aperture builds the transaction header, attaches a transport-bypass flag, builds the network header, and frames the packet for the wire — five stages cold. No work-queue entry, no completion entry, no sequence-number allocation (the bypass flag tells the receiving NIC to skip its transport-layer state machine), no retransmit-buffer slot (the CPU re-issues the load on timeout, which is cheaper than transport-layer reliability on a path that is synchronous anyway). The path admits only operations whose semantics survive transport bypass — small loads, stores, and same-path atomics — and the

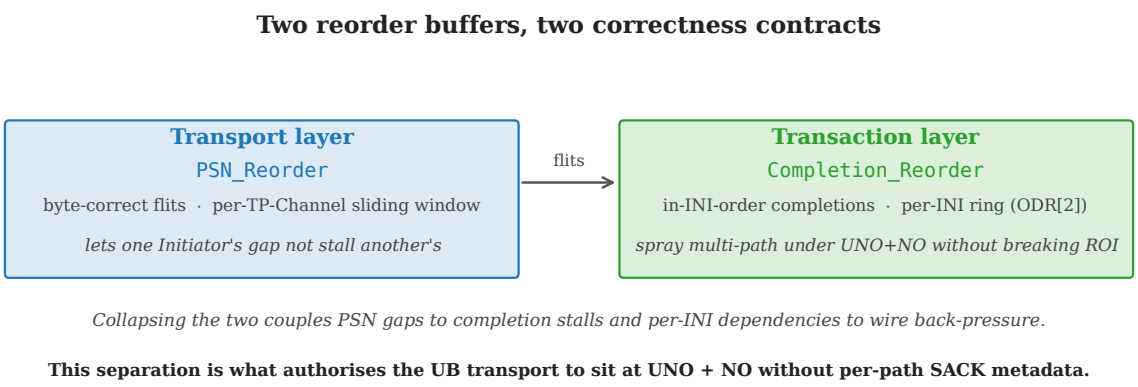


Figure 6: The pipeline carries two reorder buffers serving disjoint correctness contracts. Packet-sequence reordering at the transport layer delivers byte-correct flits regardless of application ordering; transaction-sequence reordering at the transaction layer delivers in-issue-order completions regardless of wire-byte order. The separation is what makes multi-path spreading and opt-in ordering composable.

controller must sit on the CPU's on-chip-bus segment; bulk and asynchronous transfers stay on the work-queue path.

3.4 Hardware fanout: target-side dispatch

Bounded state has a second consequence: target-side demultiplexing — which local application receives an incoming SEND — can live in hardware rather than in a CPU event loop. RoCE handles this in software (the target NIC writes a completion to a shared queue and the application event loop dispatches at CPU speed). UB defines a *Jetty Group* in which a single externally visible identifier represents a set of member Jetties; the receiving NIC dispatches each incoming SEND to a member in hardware under one of three policies (hash on an initiator-supplied key, round-robin, or receive-queue-depth load balancing) (Figure 7). The target CPU is no longer on the SEND fast path. The cost: one extra RX pipeline element and ∼80 B per 8-member group. §7.11 measures the saving at ∼200 ns per SEND for small fanout, growing to ∼1200 ns once RoCE's QP spill compounds with the CPU dispatch.

3.5 What the implementation makes visible

The implementation puts numbers on the dependencies the introduction asserted. The on-chip controller needs bounded state: with per-Jetty and per-TP-Channel tables fitting in ∼110 KB at $(1024, 1024)$, every load/store reaches the relevant context in on-chip BRAM; were the state to spill to host DRAM, the controller would pay a host-memory access on every operation and lose its latency edge. Opt-in ordering needs the layer split: the gating logic indexes per-Jetty counters that the table already provisions, costing ∼13 KLUT and zero pipeline cycles on operations that bypass gating; a QP-centric design would have to add those counters from scratch on every operation. Multi-path spreading needs the two-reorder-buffer split: the transport- and transaction-layer reorder buffers (§3.2) carry disjoint contracts, so an unordered application receives unordered completions while an ordered application sharing the same wire receives ordered ones; collapsing the buffers would force one contract onto both, and the unordered mode could no longer authorise spreading. The next sections quantify what the resulting design costs (§4) and what it saves (§5 onwards).

4. Implementation

This section opens the implementation to the level of detail the design discussion deferred: the pipeline decomposition, the two new elements for the load/store path and target-side fanout, the toolchain extensions, the SystemC and gem5 simulation infrastructure, and the timing-closure sweep that produced the 322 MHz post-route result. We describe what each piece does rather than what its source file is named; source identifiers shift over time, but the protocol responsibilities they implement do not.

OpenURMA is ∼3.5 KLOC of element-source across 39 pipeline elements covering the transport and transaction layers. The parallel OpenRoCE baseline (RoCEv2 RC) is ∼1.3 KLOC across 21 elements. Both stacks sit on the same OpenClickNP toolchain [25], so all comparisons are between implementations produced by an identical lowering pipeline.

4.1 Pipeline decomposition

The 39 OpenURMA pipeline elements break into six functional categories. The counts and responsibilities are what the protocol requires.

- **10 header parsers and builders** handle the four layered headers the spec defines — Ethernet framing, a network-transport header, a reliable-transport header, an unreliable-transport variant, and a base-transaction header — as matched parser/builder pairs.

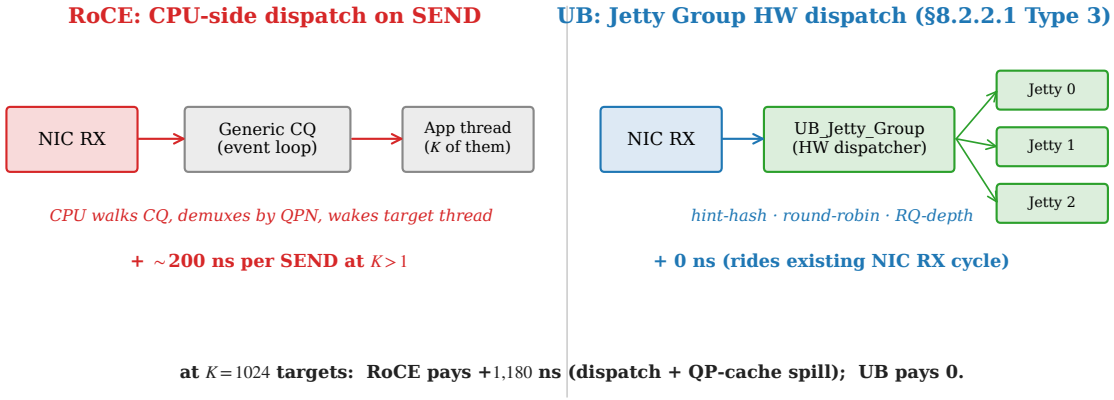


Figure 7: Target-side SEND dispatch. RoCE demultiplexes through a shared completion queue plus an application event loop; UB's Jetty Group performs the dispatch in hardware on the membus crossing already paid for by NIC RX.

- **9 transport-layer elements** implement the per-host TP Channel pipelines on both ends, the retransmission buffer, the packet-sequence reorder buffer, the two acknowledgement-generation elements (one for the lower layer's per-packet acks, one for the upper layer's transaction-level acks), the congestion-control window, the congestion-echo feedback element, and the multi-path transport-protocol-group dispatcher.
- **4 ordering elements** implement the gating surface introduced in §3.2: the Jetty scheduler at admission, two order trackers (one each at the initiator and the target, because the gating responsibility shifts between endpoints depending on the service mode), and a transaction-layer completion reorder buffer.
- **7 data-path elements** handle on-NIC memory reads and writes, atomic operations, per-Jetty receive logic, the new target-side dispatcher (§4.2), the transaction dispatch-by-opcode router, and the completion generator.
- **3 state tables** hold the memory-region permissions, the per-Jetty state, and the per-TP-Channel state.
- **6 glue elements** provide the doorbell, dispatch multiplexers, completion notification tees, completion-queue streamers, and the retransmission RTO timer.

Three pipeline-decomposition choices deserve note. The initiator-side and target-side order trackers are distinct elements because the gating responsibility moves between endpoints depending on the service mode (the relaxed mode places it at the target; the strict mode places it at the initiator). The transaction-layer completion reorder buffer is a distinct element rather than a flag on the per-Jetty receive logic because the in-order-versus-arrival-order choice is per-operation and requires a per-application sliding window, not a per-Jetty mode bit. The packet-sequence reorder buffer is distinct from the transaction-layer completion reorder buffer for the two-buffer reason discussed in §3.2: the two serve disjoint correctness contracts on disjoint sequence-number spaces.

The transmit path threads through ten pipeline stages cold for the work-queue-driven path: doorbell admission, Jetty scheduling, initiator-side ordering check, base-transaction-header build, TP-Channel transmit logic, congestion-window check, retransmission buffer write, reliable-transport-header build, network-transport-header build, and Ethernet encapsulation. The receive path inverts the sequence. The full topology is in Figure 5.

4.2 Two new pipeline elements

Two pipeline elements required new design.

The load/store transport-bypass engine. The five-stage transport-bypass pipeline (§3.3) is one new element threaded into a separate topology variant. The transmit handler accepts a load or store doorbell from the on-chip bus, allocates a one-shot transaction context (no sequence-number allocation, no retransmission slot), stamps a transport-bypass flag in the next-layer-protocol field so the receiver skips its transport state machine, and frames the packet. The first wire flit emerges at **8 cycles** ≈25 **ns at 322 MHz** — 17 cycles (≈53 ns) below the work-queue-driven path. The saving is the protocol-layer instantiation of spec-mandated transport bypass, not microarchitectural.

The target-side hardware dispatcher. This element (§3.4) sits between transaction dispatch and per-Jetty receive on the RX path. When an incoming SEND addresses a registered Jetty-Group identifier, it rewrites the destination field to a member Jetty under one of three policies: *hint-hash* (the application-supplied hint modulo the member count), *round-robin*, or *receive-queue-depth load balancing*. SENDs to unregistered identifiers pass through, so the element is opt-in per group. Per-group state is ~80 B at maximum (8-member) membership: one identifier and two counters per member plus a small header.

4.3 Retransmission and congestion control

The retransmission buffer is a 64-slot per-TP-Channel ring with both Go-Back-N and selective-retransmit modes. The selective path uses the spec-defined selective-ack response opcode plus a per-PSN bitmap in the acknowledgement header; the retransmit decision is driven either by a transaction-layer ack that explicitly NAKs PSNs (selective) or by the RTO timer (Go-Back-N). Each slot captures one metadata-and-extension flit pair, sufficient for control-plane operations and small ($\leq$8-byte) Writes. *Multi-flit retransmission* — a payload-flit list per slot so dropped multi-flit Writes can replay in full — runs on the loss-free data path but is not yet in the retransmit buffer (§10).

The congestion-control loop combines a host-side switch model with a per-channel additive-increase / multiplicative-decrease window. The modeled switch holds a queue with low/high watermarks and stamps the forward-explicit-congestion-notification bit when occupancy crosses the high watermark; the initiator-side congestion-window element multiplicatively decreases on marked acks and additively increases per RTT otherwise. A closed-loop test posts 200 work requests through the switch: 9 of the 25 packets are marked, and the sender's window drops from 65,536 B to 4,096 B.

4.4 Toolchain extensions

Six pipeline elements needed pipeline-initiation-interval guidance to close 322 MHz; the back-end previously hard-coded II=1, which over-pipelines elements whose data dependencies cannot be unrolled aggressively. We added two pragma blocks to the element-source grammar: one that lets an element declare its initiation interval (II) explicitly, and one that lets an element forward arbitrary HLS pragmas verbatim. Both extensions are upstream in OpenClickNP and reused unchanged by both stacks. They are the only modifications we made to the framework.

4.5 Three back-ends from one source

OpenClickNP lowers each element through three back-ends from one source: a thread-and-FIFO software emulator (for correctness tests, §5); a cycle-accurate SystemC simulator with a 1 ns clock matching the 322 MHz synthesis target, so SystemC cycle counts are directly comparable to post-route timing (for the microbenchmarks of §6); and a Vitis HLS back-end whose synthesisable C++ Vivado places and routes (for the LUT, BRAM, and timing numbers of §9). An element that closes timing in HLS corresponds exactly to the element the emulator tested and the simulator benchmarked.

4.6 Two-node SystemC simulator

The two-node simulator links each NIC stack as a static SystemC library and wires two instances of each side through a SystemC EtherLink module with configurable bandwidth and propagation delay. Each endpoint is a full system: a CPU host model that constructs work-queue entries and posts doorbells, the NIC's transmit and receive pipelines, a DDR4 DRAM model for both work-queue-side and data-side accesses, a two-level write-back / write-through / uncached cache hierarchy for the host CPU, and a completion-queue write-back path. The harness produces a 388-configuration sweep covering all six verb categories, four cache policies, payload sizes from 8 B to 4 KB, and link delays from 100 ns to 10 μs.

4.7 gem5 full-system scaffold

For the CPU-to-remote-DRAM end-to-end claim, we integrate the SystemC NIC into gem5 as a memory-mapped device using gem5's standard `Gem5ToTlmBridge<512>` infrastructure. A `NICTopologySC` SimObject (deriving from `sc_core::sc_module`) wraps the auto-generated 38-module TLM topology and exposes three TLM sockets as gem5 Python ports: an MMIO target for the doorbell aperture, a wire-RX target, and a wire-TX initiator. A small `WireLoopback` SimObject (pure SC, no gem5 bridges) forwards `wire_tx` flits back into `wire_rx` for single-NIC self-loop tests. The Python config wires `membus` → `Gem5ToTlmBridge512` → `nic.mmio_socket` and the SC-to-SC loopback. Doorbell flit assembly is handled in `NICTopologySC::mmio_b`: AArch64 issues eight 8-byte stores per 64-byte WR slot, which the handler aggregates into a single TLM payload before firing the doorbell.

Three load-bearing changes beyond the standard pattern were required. *First*, the SystemC libraries must be rebuilt against gem5's embedded SystemC headers, which are ABI-incompatible with the upstream Accellera build the cycle-accurate back-end normally uses. *Second*, the NIC's address aperture must be carved out of system memory because gem5's memory-bus crossbar cannot route overlapping ranges, and the SimObject's `before_end_of_elaboration` hook must explicitly call the crossbar's range-change notifier so the bus latches the new range before the binary loader's first write. *Third*, the OpenClickNP TLM backend's per-cycle drain events (scheduled via `sc_event::notify`) do not fire under gem5's atomic-CPU mode because `recvAtomic` returns without yielding to the event queue. The compiler therefore emits a public `tick_drain()` entry point on every kernel module and a topology-wide `drain_synchronous()` walker that runs every module until idle; `NICTopologySC` invokes this from every `b_transport`, delivering "equivalent to an outer `sc_start(N)`" semantics without entering the SC scheduler from inside the bridge.

A dual-node FS-mode configuration brings up an ARM CPU running Linux 4.14, the `uburma.ko` character-device driver, and a libc-linked `urma_smoke` benchmark that exercises the three CQE-delivery paths reported in §5. The full WRITE → TAACK → CQE roundtrip required fixing three SC-pipeline gaps the synthetic-injector era had masked: (a) the wire encoder/decoder did not preserve `svc_mode`, so the responder's TAACK generator

dropped ROL/UNO packets unconditionally — fix: encode `svc_mode` in BTAH wire byte $b[5]$ bits 0–1; (b) the receive-side `PSN_Reorder` stranded the EXT flit when its META drained in-order with an empty companion slot — fix: forward the EXT directly to port 1 when no waiting OOO slot needs it; (c) the initiator-side `rtph_p[2]` (received TPACK) port was wired to `Drop` as MVP — fix: re-route to `cqe_stream` so each received transport-level ACK becomes a host CQE.

4.8 Timing-closure sweep

The initial Vitis-HLS pass on the 38 work-queue-driven elements left 8 elements needing remediation: 5 missed 322 MHz in Vivado P&R and 3 failed at HLS itself before reaching P&R. The mechanical resolution combined the new II and HLS-pragma extensions described above with a few targeted algorithmic redesigns, summarised by element function in Table 1.

Element function	Before → After (ns WNS)	Action
Atomic operations	$-1.871 \rightarrow +0.298$	II=4 + mem partition
Completion reorder	HLS-stuck → $+0.672$	head-ring + II=2
Initiator order tracker	$-5.382 \rightarrow +0.318$	II=2
Jetty scheduler	HLS-aborted → $+0.546$	II=2
Target order tracker	$-2.25 \rightarrow +0.101$	II=2 + drain reorder
On-NIC memory write	$-0.628 \rightarrow +0.628$	memory partition
Memory-region table	failing → $+0.729$	shrink to 64 entries
On-NIC memory read	$-0.449 \rightarrow +0.282$	word-sized array

Table 1: Timing-closure remediation, by element function.

One instructive failure. The most instructive case is the on-NIC memory-read element, which originally backed its 64 KB read path with a byte-sized array plus cyclic memory partition for parallel byte-bank access. HLS estimated 490 MHz F_{max}; Vivado P&R reported -0.449 ns WNS. The worst path ran from a register inside the per-bank index computation into the BRAM address port of the partitioned array: 8 LUT levels of barrel-shift mux to compute the per-bank index, plus 2.7 ns of routing delay (76% of the period). No combination of II or partition factor moved the slack; the wide muxing for the 8 banks bloated routing without unblocking timing. The fix was algorithmic: replace the byte-sized array with an 8-byte-word-sized array indexed by a shifted offset. Each 8-byte-aligned read becomes a single BRAM word access; the byte-bank mux is gone; the memory partition pragma is no longer needed. WNS closes at $+0.282$ ns. The same pattern was applied to the OpenRoCE on-NIC memory-read element. The lesson is that HLS's per-element timing estimate is a useful but optimistic predictor; routing-bound paths only show in Vivado P&R, and the right resolution may be at the data-layout level rather than the pragma level.

5. Evaluation methodology and functional correctness

Without physical FPGA, the evaluation runs the three back-ends of §4.5 as three independent measurement paths: the **software emulator** answers the correctness questions; the **SystemC simulator** reports cycle-accurate latency, throughput, and ordering cost; and the **Vivado out-of-context flow** reports post-route LUTs, flip-flops, BRAM, and worst-negative-slack against a 3.106 ns target. All three back-ends consume the same element-source, so a kernel that passes correctness also runs in SystemC and HLS; every number in the rest of the paper is reproducible from a single evaluation harness that writes CSV.

A 17-test software-emulator integration suite covers the spec surface and framework internals; all pass on the current tree.

- **Initiator-side ordering** — a strict-order operation is gated behind a relaxed-order operation; the pipeline holds the strict op at the initiator until the relaxed one commits.
- **Target-side ordering** — the target serialises strict-order operations at execution time, deferring them behind missing prior operations.
- **Fused acknowledgement** — the lower-layer ack carries the upper-layer ack on the same flit, saving one wire packet per response under the fused-ack service mode.
- **Fence** — a fenced operation waits until all prior reads and atomics have responded before its own emission.
- **Completion ordering** — with the in-issue-order bit set, completions are retired in original-issue order; with it clear, they are retired in arrival order.
- **Unordered Send wire path**.
- **Mixed-mode operations** in the same send queue.
- **Multi-initiator parallelism** — per-initiator gating: a strict-order operation on one initiator emerges immediately while a strict-order operation on a different initiator is gated behind its own dependency.
- **Head-of-line-blocking isolation** — 8 unordered operations from 4 initiators all emerge despite a stalled strict-order operation on a separate initiator.
- **On-NIC memory read/write data integrity**.
- **Full atomic opcode set** — swap, store, load, fetch-and-add, fetch-and-subtract, fetch-and-and, fetch-and-or, fetch-and-xor; each verified to return the pre-modification value in the response and to leave the on-NIC memory word in the algebraically correct post-state. Compare-and-swap is covered by the mixed-mode test.
- **Transmit-to-wire round trip** with all UB header fields preserved.
- **Sustained-throughput sanity** (50,000 work requests).
- **Wire-format encoder** against the spec bit layout.
- **End-to-end congestion-control loop** — 200 work requests posted; 9 of the 25 packets that traverse the modeled switch are congestion-marked; the sender's congestion window backs off from 65,536 B to 4,096 B.
- **Target-side hardware dispatch** — the dispatcher routes incoming SENDs to one of N member Jetties

under three policies (hash on an application-supplied hint, round-robin, receive-queue-depth load balancing). The test exercises all three on a 4-member group plus the unregistered-identifier bypass; the end-to-end consequence is quantified in §7.11.
- **Multi-flit Write payload path** — 8/32/64/128/256-byte writes through Ethernet-encapsulation streaming, decapsulation reassembly, and the on-NIC memory-write payload byte stream.

What the SW-emu suite does *not* cover. The eval as it stands exercises every implemented mode and opcode on the loss-free path. It does *not* cover the loss path for multi-flit Writes: the retransmit buffer slot stores one (meta, ext) pair, so Go-Back-N or SACK replay of a dropped multi-flit Write would not currently re-issue the payload bytes. Enabling that requires extending the retrans slot to a payload-flit list (a follow-on work item, §10). All single-flit retransmit paths (control-plane WRs, $\leq$ 8 B Writes, Reads, Atomics) are exercised by the C-AQM closed-loop test through the congestion-echo element. Explicit packet-drop injection at the SW-emulator level remains follow-on work; the loss-rate sweep in §8.5 exercises drop injection at the two-node simulator level.

6. Cycle-accurate microbenchmarks

We instrument the TX pipeline (the harder of the two paths to reason about, since it is where the ordering surface sits) under several SystemC scenarios. Each scenario runs as one SystemC microbenchmark invocation and writes one CSV row per parameter point.

6.1 Per-stage latency breakdown

A single Write WR (ROL+NO) drives the pipeline cold. Tap monitors inserted between every adjacent stage record the first-flit arrival time. Figure 8a shows the per-stage contribution.

The slowest stages are the Jetty scheduler (5 cycles — II=2 plus the per-initiator round-robin scan) and Ethernet encapsulation (11 cycles — byte-stream-rate, not flit-rate). The initiator-side order tracker costs 1 cycle on the unblocked path: the gating logic adds combinational depth (covered in §9) but no extra pipeline cycles when no prior dependencies are outstanding. Total cold-path TX latency: 24 cycles $\approx$ 75 ns at 322 MHz.

6.2 Per-mode TX latency

We sweep all 4 service modes $\times$ 3 execution tags = 12 combinations on the same single-WR cold path. *Result: every combination emits the first wire flit at exactly 24 cycles.* The per-mode penalty is therefore not in pipeline-stage count but in the combinational logic depth of each kernel — and that is what shows up in the Vivado area report (§9), not in the SystemC cycle count. This is the design's intended behaviour: opt-in ordering does not add pipeline cycles when not used.

6.3 Sustained throughput per service mode

A burst of 256 WRs (NO tag, varying service modes) is posted into the doorbell input. Steady-state throughput is calculated as $(N-1)/(\Delta t)$.

Service mode	Steady WR/μs
OpenURMA: ROI / ROT / ROL / UNO	**150.36**
OpenRoCE RC (matched microbench)	53.62

All four OpenURMA service modes share a single transmit pipeline (the unordered mode bypasses sequence-number allocation within the TP-Channel transmit element but still traverses the same kernel chain), and the throughput floor is set by the slowest pipeline initiation interval in the chain (II=2 in the Jetty scheduler, the initiator-side order tracker, the completion reorder buffer, and the target-side order tracker; II=4 in the atomic-operation element). Hence the per-mode rate is identical at 150.36 WR/μs. The OpenRoCE baseline runs the matched-shape microbenchmark on identical infrastructure and measures 53.62 WR/μs — **2.80$\times$ slower** — because RC's per-QP sequence-number allocation introduces stricter inter-operation dependencies in the QP transmit kernel. The 1000-WR burst-saturation regime reaches 159.46 WR/μs on OpenURMA and 53.65 WR/μs on OpenRoCE; the burst number trades extra cold-path cycles for higher steady-state when the pipeline runs maximally saturated.

6.4 Ordering surface: Fence and serialised-order gating cost

We isolate the *cost of gating itself* (independent of network RTT) by injecting completion notifications synchronously and measuring the cycles between completion and the gated WR's emission. Figure 8b plots both costs.
- **Fence (spec §7.3.2.2 note).** Posting a Fenced Write behind N pending Reads, with all N completions notified simultaneously after a 30-cycle delay: the Fenced WR emerges 4–48 cycles after notification, scaling near-linearly with N (the Jetty scheduler's round-robin scan visits each initiator in turn at 1 step/cycle).
- **SO (initiator-side order tracker, spec §7.3.3.2).** Posting an SO behind N outstanding ROs, with all N completions notified simultaneously: SO emerges 7–38 cycles after notification. The cost is bounded by the initiator-side order tracker's 16-position round-robin cursor.

These costs are small (under 50 cycles across the 0–4 outstanding range we sweep here; the per-initiator queues are sized 32 so they would not saturate even at much deeper outstanding counts). Crucially, they are paid only when gating is requested; an application that uses NO+UNO sees neither.

6.5 Cross-initiator head-of-line isolation

We force one initiator into a permanent ROI+SO stall by emitting an RO and then a gated SO whose RO completion is never notified. We then post 8 UNO+NO WRs

from four different initiators. Figure 8c shows that all 8 unblocked WRs emerge at the wire within 78 cycles of post — they bypass the gating elements completely. The stalled SO does not propagate back-pressure across the per-initiator queues. This is the key ordering-surface guarantee that is impossible under RC: head-of-line on one connection does not block another.

6.6 Throughput vs payload size

Figure 8d sweeps payload from 8 B to 4 KB. The per-WR rate is essentially constant at $\approx$141 WR/μs, confirming that the pipeline is metadata-flit-rate-limited at small-to-medium payloads — exactly the regime that matters for AI collective control packets and HPC small-message all-to-all. At 4 KB payload (129 wire flits per WR), the same WR rate translates to a wire-byte rate of $\approx$4.6 Tb/s — well above the U50's single-port CMAC line rate, indicating that bandwidth is not the binding constraint at any payload size in this regime.

6.7 Load/store transport-bypass cold path

The microbenchmarks above exercise the work-queue-driven path, which traverses ten pipeline stages cold (doorbell admission, Jetty scheduling, initiator-side ordering, base-transaction header build, TP-Channel transmit, congestion-window check, retransmission-buffer write, reliable-transport header build, network-transport header build, and Ethernet encapsulation). The load/store path replaces all ten with five: a CPU load or store reaches an on-chip-bus-attached controller, the controller flags the request for transport bypass, and the packet emerges through the header builder and encapsulator. The transport layer is omitted entirely (no sequence-number allocation, no retransmission slot, no transport-layer header). Posting a single load request, the first wire flit emerges at **8 cycles $\approx$ 25 ns at 322 MHz** — 17 cycles ($\approx$53 ns) below the work-queue-driven cold path on identical infrastructure. The saving is structural, not microarchitectural; it removes the transport-layer state machine from the NIC budget for the workloads that opt in.

Why load/store, specifically, admits transport bypass. The restriction is structural. Atomic operations need remote serialisation (a second fetch-and-add must observe the first's commit), and the transport layer is what provides that order. Read and Write route through the work-queue path to preserve completion-queue semantics for the asynchronous verb surface. Load and store, by contrast, carry no inter-operation ordering contract at the application level — the CPU's load-use dependency chain already provides the order the transport layer would otherwise enforce — so the transport layer is redundant and elides.

Is 8 cycles a substrate floor? Yes: each of the five pipeline stages already runs at II=1, and the remaining 3 cycles are state-machine setup at the boundaries. Collapsing two stages would push the critical path through two parsers in one cycle and miss the 3.106 ns period — the same routing-density wall the on-NIC memory-read story (§4) formalises. An ASIC at 322 MHz is still 8-cycle-floored; an ASIC at 1 GHz yields 8 ns absolute, and tighter cell delays let adjacent stages collapse to 5–6 cycles. He's [11] $\sim$100 ns end-to-end UB ASIC number is consistent with this budget ($\sim$5–8 ns each NIC + link + DRAM).

7. Two-node end-to-end evaluation

The microbenchmarks in §6 measure the NIC pipeline alone. The architectural argument UB attaches to memory-semantic access (He [11], spec §8.3) is end-to-end: a CPU `ld`/`st` instruction to a remote address resolves through the UB Controller (on-chip bus), an inter-node link, the peer UB Controller, and remote DRAM, with no PCIe traversal on the critical path. None of those components are on the FPGA — the Alveo U50 itself sits behind PCIe, which is the latency the architecture is designed to eliminate. This section presents a two-node simulator that integrates the cycle-accurate kernels with analytical models of the surrounding components, so all four NIC-stack configurations can be compared on identical workloads.

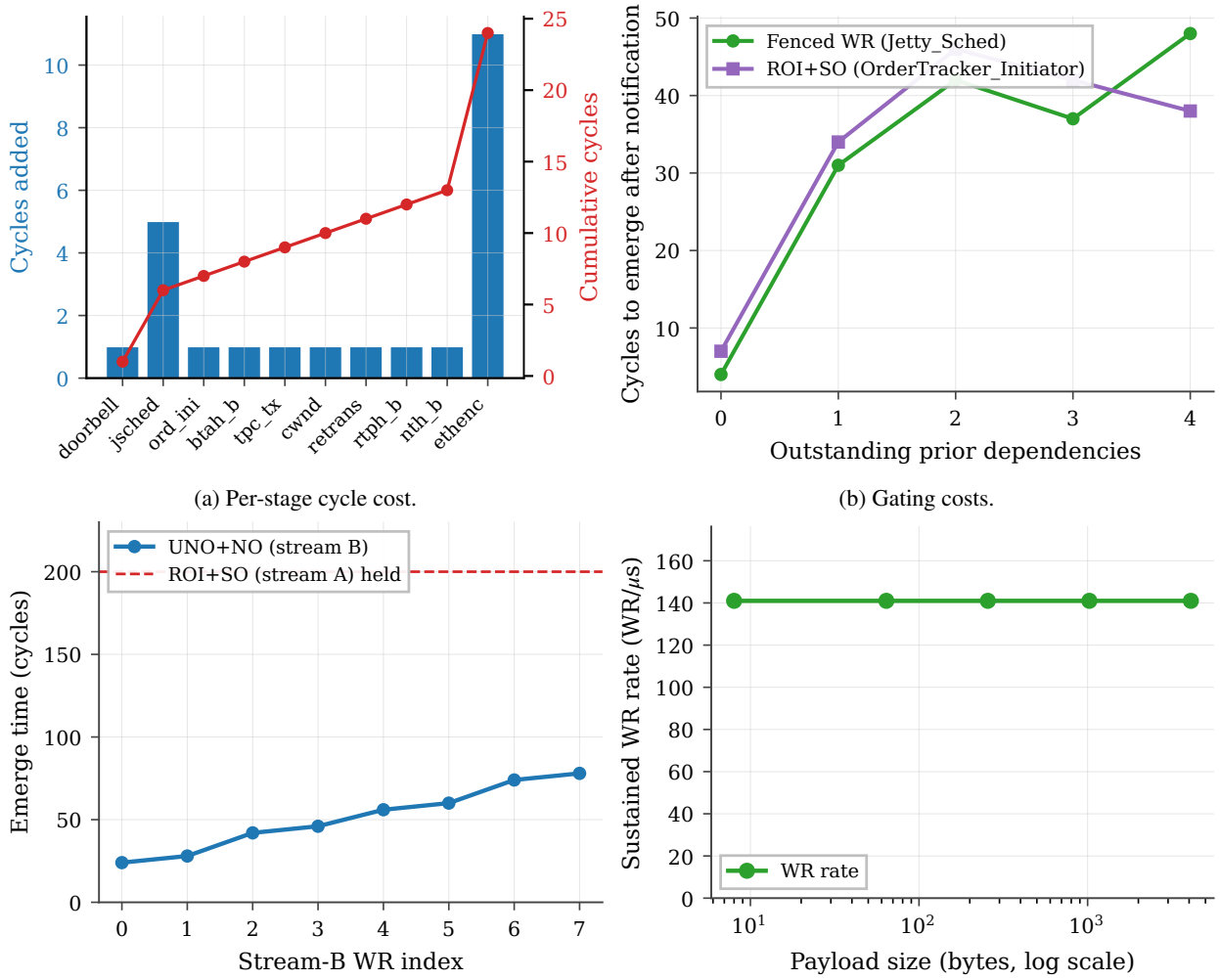
7.1 Methodology

The simulator is a SystemC program that links the two NIC RTL stacks (the OpenURMA UB stack and the OpenRoCE RC stack) as static SystemC libraries and instantiates two endpoints. Four *configurations* are exposed (and used as columns in every later table):

- **UB LD/ST** — the spec §8.3 load/store transport-bypass path through the OpenURMA stack.
- **UB URMA** — the spec §8.4 work-request path through the OpenURMA stack (the verb-driven path; "URMA" is Huawei's branding for the UB verb library).
- **RoCE BF** — the OpenRoCE RC stack with BlueFlame inline-WQE (*i.e.*, the doorbell carries the WQE inline when WQE $\leq$ 64 B, eliminating the DMA WQE fetch).
- **RoCE DMA** — the OpenRoCE RC stack with the standard DMA-fetched WQE path (the WQE lives in host DRAM and the NIC reads it back over PCIe).

BF and DMA are runtime modes of the same RoCE stack, not two separate stacks; LD/ST and URMA are two separate top-level topologies of the same OpenURMA pipeline. Each NIC transmit/receive cycle count is taken directly from the corresponding cycle-accurate SystemC microbenchmark at 322 MHz: 8 for UB LD/ST, 25 for UB URMA, and 9 for RoCE RC (both modes share the same NIC pipeline).

Decomposed cost model — full bidirectional accounting. Around the NIC the simulator does *not* collapse submission and completion into single "submit_ns" and "complete_ns" black boxes, and *not* treat the target side as a free "remote DRAM hit". Every host$\leftrightarrow$NIC interaction on *both* sides of the wire is an explicit SystemC



(a) Per-stage cycle cost.

(b) Gating costs.

(c) Cross-INI HOL bypass.

(d) Throughput vs payload.

Figure 8: SystemC microbenchmarks. (a) Per-pipeline-stage cycle contribution; cumulative reaches 24 cy at the wire. (b) Cost of Fence and SO gating in cycles after completion notification. (c) UNO+NO WRs from four initiators emerging at the wire while a separate initiator's ROI+SO is indefinitely held — the stalled SO does not propagate back-pressure. (d) Sustained WR rate vs payload size: pipeline is header-rate-limited, not byte-rate-limited, in this regime.

component on the critical path. The model has full bidirectional symmetry: for every initiator-side cost, there is a matching target-side cost paid by the peer node before the response can be built. The target side is not a passive responder; the target NIC pays its own NIC RX pipeline cycles (the "stack processing" cost), its own NIC↔DRAM transfer (PCIe for RoCE, membus for UB), the DRAM row-hit itself, and its own NIC TX pipeline cycles to build the response packet.

- **Verb-library post overhead** (50 ns) — user-space verb library overhead on the post-send call: function dispatch, send-queue validation, a store-ordering memory barrier, lock acquisition. Zero for the load/store path (the verb IS the ISA instruction; no library call).
- **Work-request construction** (30 ns) — CPU stores building the work-request structure in host memory. Zero for the load/store path (no work-request structure).
- **Doorbell MMIO** (150 ns) — posted PCIe write to the NIC's MMIO BAR. Zero for UB (on-chip bus).
- **Work-queue-entry DMA fetch** (500 ns) — NIC-initiated PCIe DMA read fetching the work-queue entry from host DRAM. Zero for UB and for the RoCE inline-WQE path with WQE $\leq$ 64 B.
- **On-chip-bus traversal** (30 ns) — controller on-chip-bus traversal, used for both submission and completion on the UB stacks.
- **Completion-entry DMA write** (250 ns) — NIC-initiated PCIe DMA write posting the completion entry to host DRAM. RoCE only.
- **Completion-entry poll** (70 ns over PCIe, 5 ns on-chip) — CPU spin-load of the completion record. Zero for the load/store path (synchronous; the load returns directly to a register).
- **Verb-library overhead** (30 ns) — the user-space verb library walks the completion queue, marshals the work-completion structure, and returns to the caller. Zero for the load/store path. Applied symmetrically to UB's work-request path and to RoCE so the software-side cost is not asymmetric between the WR-based stacks.
- **Target-side NIC–DRAM transfer** — this is the cost the "single-node" RDMA latency literature most often skips. The target NIC must move the operand between

itself and the target host DRAM *before* (for READ and atomic operations) or *after* (for WRITE and SEND) the local DRAM row access. RoCE pays a PCIe DMA on every operation (500 ns for READ/atomic, 250 ns for WRITE/SEND); UB pays only the on-chip-bus crossing (30 ns) because the controller is on the on-chip bus on the target as well.

- **Initiator-side response DMA** — for READ-class verbs, the initiator-side payload DMA *after* the response packet arrives (the NIC DMA-writes the returned payload to host DRAM, separately from the completion-entry write). 250 ns for RoCE; zero for UB (the payload returns on-chip).

Figure 9 sketches each stack's round trip — which traversals are PCIe (dashed) and which are on-chip or wire (solid) — and Table 2 consolidates the full per-side decomposition for a 64 B READ. Every row in the table is paid on the critical path before the next phase can start.

The simulator total matches the per-row sum to within SystemC inter-thread scheduling overhead — 14 ns for RoCE DMA, 12 ns for UB URMA, and 80 ns for UB LD/ST. The larger delta on the LD/ST path is because that path's total is dominated by a single chain of three short ($\leq$30 ns) SystemC components (membus, NIC, wire) with no DRAM or DMA stage to amortise the per-handoff fixed-cost; absolute scheduling overhead is similar across stacks but represents a larger fraction of the 420-ns LD/ST total than of the 2172-ns RoCE total.

Table 2: Per-side latency decomposition (ns) for a READ verb, 64 B payload, link delay 100 ns, polling completion. Every row on the table is paid on the critical path. The "Target NIC↔DRAM" row is the cost the single-node-style RDMA latency literature typically omits.

Phase	UB LD/ST	UB URMA	RoCE DMA
Initiator side, pre-wire			
Verb library post	0	50	50
WQE construct	0	30	30
Doorbell MMIO	0	0	150
DMA WQE fetch	0	0	500
Submit (membus)	30	30	0
NIC TX pipeline	25	78	28
Wire forward	100	100	100
Target side, between wire and DRAM			
NIC RX (stack proc.)	25	78	28
Target NIC↔DRAM	**30**	**30**	**500**
Target DRAM row hit	30	30	30
NIC TX (response)	25	78	28
Wire back	100	100	100
Initiator side, post-wire			
NIC RX (response)	25	78	28
Initiator resp DMA	0	0	250
DMA CQE write	0	0	250
Complete (membus)	30	30	0
CQE poll	0	5	70
Verb library poll	0	30	30
Total (modeled)	**420**	**745**	**2172**
Total (measured)	**500**	**757**	**2186**

Defaults follow published ConnectX-7-class ranges [39, 21]; every value is exposed as a command-line knob so the comparison can be repeated under different parameter assumptions. We model only polling completion — the regime high-performance RDMA uses — and deliberately omit event-driven completion: the latter requires modelling MSI-X delivery + kernel interrupt-handler dispatch + scheduler decisions + thread wakeup, all of which are OS-dependent and cannot be faithfully represented by a single analytical parameter; FastWake [28] characterises that host-side path directly. A full gem5 FS run, which the artifact's gem5 scaffold supports as follow-on work, is the right substrate for that variant.

CPU-side cache hierarchy. A two-level cache (L1, L2) + LLC + local DRAM hierarchy in the simulator is consulted on every §8.3 `ld`/`st`. Fully-associative LRU replacement, three policies (write-back / write-through / uncacheable), default hit latencies L1: 1 ns, L2: 4 ns, LLC: 12 ns, local DRAM: 70 ns. §8.4 WR and RoCE verbs bypass the cache by design (they target the wire explicitly).

Wire and remote DRAM. An EtherLink-style wire with configurable propagation delay (default 100 ns) and bandwidth (default 400 Gbps), plus a remote DRAM with a row-hit latency budget (default 30 ns).

Verb coverage. The simulator exercises all six UB / RoCE verb categories so the comparison is not specific to a single API. Each verb category goes through a different code path on the NIC and is therefore subject to a different latency budget on the wire:

Verb	UB path	RoCE equivalent
Load / Store	LD/ST transport-bypass	— (not supported)
Read	work-queue path	RDMA_READ
Write	work-queue path	RDMA_WRITE
Send	work-queue + receive-side	SEND
Receive	receive-side completion	RECV
FAA / CAS / Swap	atomic-set elements	ATOMIC_FAA / CAS

The simulator routes each verb through the correct egress and ingress payload model: request-only flits for Read, payload-carrying flits for Write and Send, read-modify-write at the remote ALU for atomics, and receive-side queue handling for two-sided Send/Receive.

Eight workloads.

- **Pointer-chase** — linked-list traversal, dependency-chain load-latency-bound. Parameterised by verb (load on the UB load/store path, READ on the work-queue path and on RoCE) and by locality fraction.
- **Distributed-barrier** — N-process fetch-and-add on a shared counter; a latency-bound synchronisation primitive.
- **Hash-probe** — random-key memcached-style lookup.
- **CAS-lock** — single-contended-line compare-and-swap lock acquisition.
- **Graph-BFS** — mixed READ/WRITE traversal of remote vertices ($\sim$75% READ, 25% WRITE).
- **Send/receive pingpong** — two-sided pingpong via SEND/RECV verbs, exercising the receive-side queue.

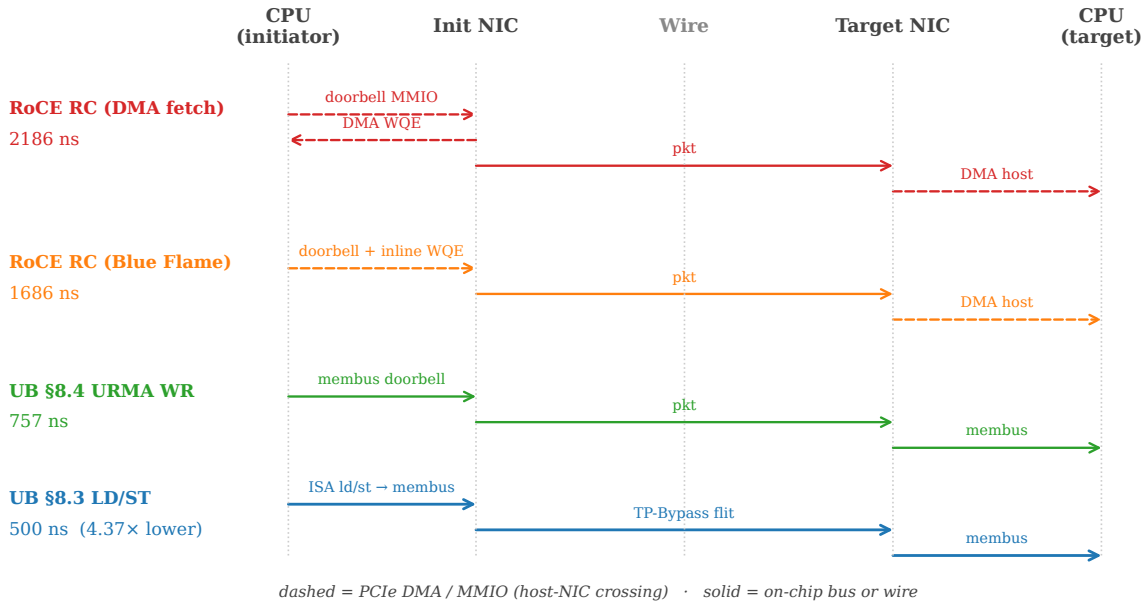


Figure 9: Submission path per stack. RoCE traversals between CPU and NIC (dashed) sit on PCIe; UB carries the same hand-offs over the on-chip bus. On UB LD/ST the verb *is* an ISA instruction, and the wire crossing carries a TP-Bypass flit instead of a full RTPH-wrapped packet.

- **Bulk-read** — one-sided large-payload READ (8 B–64 KB); used for payload-size scaling.
- **Bulk-write** — the WRITE-only dual.

Sweep matrix. NIC stack (×4) × link delay $\{50, 100, 200, 500\}$ ns × in-flight concurrency $\{1, 4, 16, 64\}$ × payload $\{8, 64, 256, 1024, 4096, 16384, 65536\}$ B × cache policy {WB, WT, UC} (§8.3 path only) × cache-locality $\{0, 25, 50, 80\}$ % (§8.3 path only). 200 operations per run, 388 valid sweep points total at the released CSV sweep.

7.2 End-to-end latency

Figure 10 reports the per-operation latency at link-delay = 100 ns, payload = 64 B (or 8 B for distributed-barrier/CAS-lock), concurrency = 1, polling completion, with target-side NIC↔DRAM DMA explicitly modeled. Headline numbers (mean, ns):

Verb (workload)	UB LD/ST	UB URMA	RoCE BF	RoCE DMA
LOAD (ptr-chase)	**500**	—	—	—
READ (bulk-read)	—	757	1686	2186
WRITE (bulk-write)	**750**	750	1177	1677
SEND (pingpong)	**804**	804	1231	1731
FAA (dist-barrier)	**750**	750	1427	1927
CAS (CAS-lock)	**750**	750	1427	1927

On the memory-semantic operation (CPU fetches 64 B from remote memory), UB executes the verb as a LOAD through the spec §8.3 load/store + TP Bypass path while RoCE must execute it as a READ through the work-request path. UB is **4.37× lower latency** than the RoCE DMA baseline (500 ns vs 2186 ns), 3.37× lower than RoCE BF (1686 ns), and 1.51× lower than UB's own URMA work-request path (757 ns). On verbs that even UB must route through §8.4 (READ, WRITE, SEND, atomics), UB still pays no PCIe and so remains **2.2–2.9× lower latency** than the matched RoCE DMA baseline. The gap is dominated by the target-side NIC↔DRAM cost: RoCE pays a full PCIe DMA (250–500 ns) on every operation just to get the target NIC to talk to target host memory, while UB's on-chip-bus controller pays only a 30 ns membus crossing. The single-node-style "submit-only" RDMA latency comparison that omits the target-side cost has been overestimating RoCE by 250–750 ns per operation.

7.3 Op-rate vs concurrency

Figure 11 shows op-rate scaling as concurrency grows from 1 to 64. UB LD/ST reaches ∼2.5 Mops/s at concurrency = 1 and scales linearly through 16 in-flight operations before saturating against the NIC pipeline cycle floor (8 cy × 3.106 ns). RoCE DMA bottoms out at ∼0.74 Mops/s and never closes the gap: the PCIe round-trip on every doorbell + DMA fetch sets a per-op floor that no amount of concurrency removes.

7.4 Sensitivity to link delay

Figure 12 plots per-op latency against one-way link delay over the 50–500 ns range. The four curves are order-preserving: UB LD/ST stays the lowest at every link delay. On the pointer-chase workload (the one plotted; a mixed- locality stream where some loads hit cache and others miss to the wire), the ratio between UB LD/ST and RoCE DMA narrows from ∼3.4× at 100 ns link delay to ∼2.5× at 500 ns as the wire term dominates the submission-side overhead. (The headline 4.37× on a cold 64 B READ in §7.5 is the worst point for RoCE,

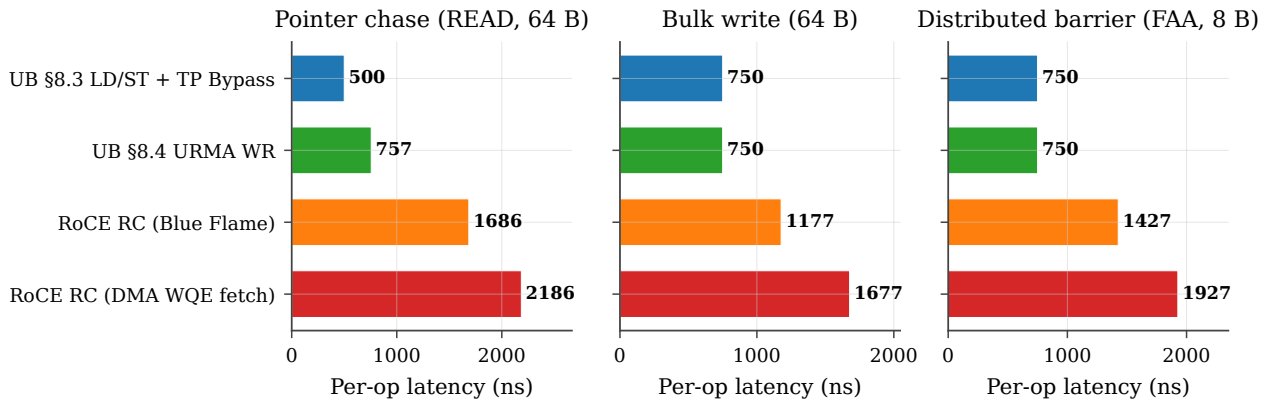


Figure 10: Per-operation latency (CDF). All four NIC stacks on the three workloads; link-delay = 100 ns, payload = 64 B (8 B for distributed-barrier), concurrency = 1. UB LD/ST is the leftmost curve in every panel.

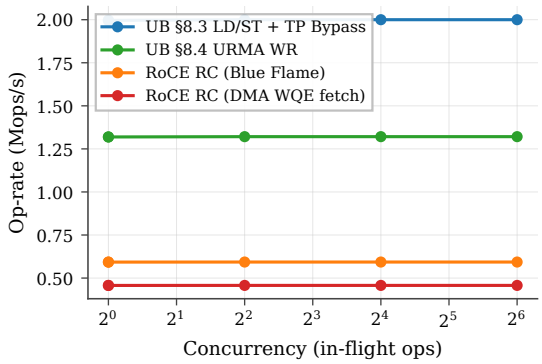


Figure 11: Op-rate vs in-flight depth on pointer-chase, link-delay = 100 ns, payload = 64 B.

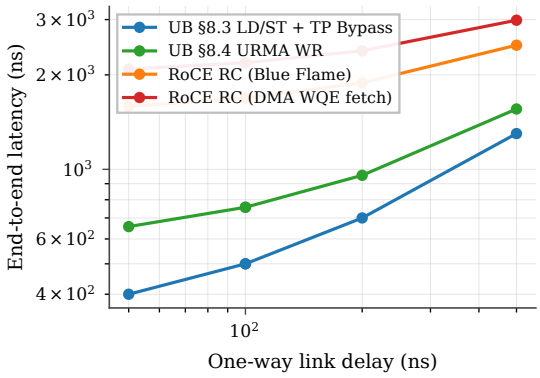


Figure 12: End-to-end latency vs one-way link delay on pointer-chase, concurrency = 1, payload = 64 B.

not the pointer-chase mean.) The absolute gap grows with link delay (956 ns at 100 ns, ≈2 μs at 500 ns), since UB LD/ST benefits from the link round-trip not being multiplicatively inflated by per-end PCIe traversals.

7.5 Decomposed latency budget

Figure 13 plots the round-trip budget for each stack at the headline operating point (link = 100 ns, payload = 64 B, concurrency = 1, polling completion). Per-row component costs are documented in the methodology itemize above and tabulated in Table 2; the figure visualises the same numbers as a stacked bar.

The two RoCE bars are dominated by five PCIe traversals on the critical path — initiator-side doorbell MMIO, DMA WQE fetch, initiator-resp payload DMA, and DMA CQE write; target-side DMA-read of host memory — which together cost ∼1650 ns, more than 5× the entire UB LD/ST budget. RoCE BF saves 500 ns on the initiator side by inlining ≤64 B WQEs but still pays the target-side DMA-read (the largest single PCIe term). UB URMA avoids every PCIe traversal because the UB Controller is on the on-chip bus on both sides, paying only $50 + 30 + 30 + 5 + 30 = 145$ ns of verb-library plus membus crossings. UB LD/ST pays none of the software-side overhead either: the host-NIC hand-off is an ISA `ld`/`st` returning to a register, and the on-chip-bus path stays inside the memory-bus fabric on both nodes.

Why the nine components vanish, not shrink. The nine RoCE-side costs Table 2 enumerates are not nine vendor-tunable inefficiencies but three protocol consequences of placing the NIC behind PCIe. Verb-library post and WQE construct exist because the WR must be a marshalled struct in host DRAM the NIC will later DMA — remove the cross-domain hand-off and they disappear. Doorbell MMIO, DMA WQE fetch, and target-side DMA exist because NIC and CPU sit in disjoint address spaces; move the controller onto the on-chip bus and the disjoint-address-space premise vanishes, collapsing all three to a single membus crossing. Initiator-resp DMA, DMA CQE write, CQE poll, and verb-library poll exist only as the cross-domain completion-notification protocol; under UB LD/ST the ISA's load-use dependency chain replaces that protocol entirely. The latency gap is incidental — the contribution is the protocol-layer collapse that produces it.

7.6 Verb coverage

Figure 14 reports mean per-op latency for seven verb classes at link = 100 ns, conc = 1, 64 B (8 B for atomics). Two patterns emerge. (i) On the memory-semantic

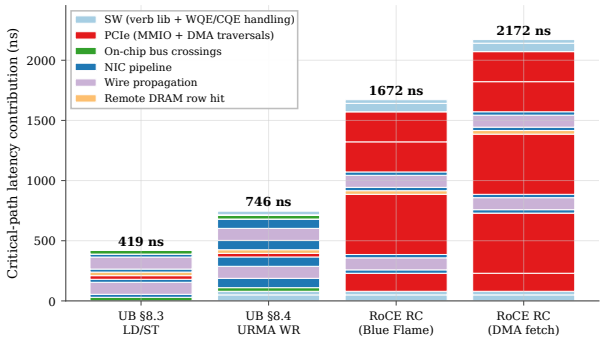


Figure 13: End-to-end latency budget by component, link = 100 ns, payload = 64 B, concurrency = 1.

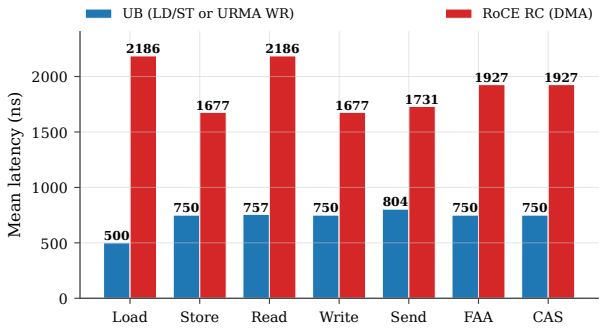


Figure 14: Per-verb mean latency comparison. UB (§8.3 LD/ST for Load/Store; §8.4 URMA WR for Read-/Write/Send; §7.4.2.3 atomics for FAA/CAS) vs RoCE RC (DMA WQE fetch). Link = 100 ns, concurrency = 1, payload = 64 B (atomics: 8 B operand).

operation (CPU fetches 64 B from remote memory, implemented as LOAD on UB §8.3 + TP Bypass and as READ on RoCE), the UB stack is **4.37**× lower latency than the matched RoCE-DMA path (500 ns vs 2186 ns). (ii) On verbs that even UB must route through the UB URMA work-request path (READ, WRITE, SEND, atomics — the spec does not authorize TP Bypass for these), the UB stack is still consistently ∼2.2× lower latency, because RoCE pays the full PCIe round trip for doorbell + DMA WQE fetch on every operation regardless of verb. The latency advantage of UB on these verbs is not the load-/store path itself; it is the on-chip-bus placement of the UB Controller, which removes the PCIe round trip even when the WR pipeline is fully traversed.

7.7 Cache policy sensitivity

The UB LD/ST path benefits asymmetrically from CPU-side caching, because cacheable hits short-circuit the remote round trip entirely. Figure 15 sweeps cache locality from 0 % (every load misses) to 80 % (the hot 32-line working set fits in L1) under three cache policies. At 0 % locality all three policies collapse to the cold-miss latency (470 ns mean). At 80 % locality write-back / write-through deliver 175 ns mean (a ∼2.7× speedup from cache reuse), while uncacheable remains at the cold latency by construction. This is the architectural reason the spec (§8.3) pairs Load/Store synchronous access with

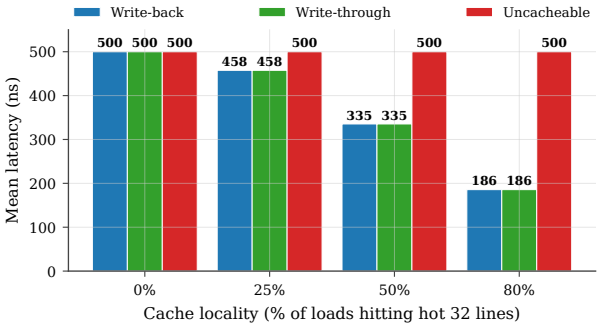


Figure 15: UB LD/ST latency under three cache policies (write-back, write-through, uncacheable) as cache locality varies from 0 to 80 %. Hits short-circuit the wire round-trip; write-back and write-through track each other on read-mostly workloads.

TP Bypass: the benefit case is precisely the workload class with non-zero cache locality, and TP Bypass minimises the cost paid on the miss path. RoCE verbs cannot exploit this — they go to the wire by design, even on cached lines — so workloads with high locality see UB's relative advantage grow further beyond the 4.37× cold-miss baseline.

TP Bypass extends the cache hierarchy through the wire. The implication runs deeper than the latency numbers. Under §8.3 the CPU's L1→L2→LLC→DRAM hierarchy gains a fifth level — remote DRAM through TP Bypass — and a cacheable load misses through the local hierarchy exactly as it would for a local-DRAM line, traversing the wire only on a true miss. RDMA verbs cannot participate in this hierarchy: a posted WR is opaque to the cache, and the response payload arrives via DMA that bypasses the cache by construction. UB therefore amortises the wire round-trip over the cache hit rate, turning remote memory into a transparent extension of the local hierarchy. Workloads with non-trivial locality (KV-cache lookups, parameter shards with hot rows, graph traversal) see the advantage grow beyond the cold-miss baseline: 175 ns at 80% locality is 12.5× faster than the 2186 ns RoCE-DMA cold miss, not 4.37×.

7.8 Page-swap baseline (Infiniswap / Fastswap)

The load/store path's value to applications is not only latency. The class of academic "infinite-swap" systems — Infiniswap [10], Fastswap [2], Leap [1], and Hermit [38] — targets the same programming model the spec §8.3 path admits: *use remote memory as a slower local memory without modifying the application*. These systems achieve it by sitting under the Linux swap subsystem and posting RDMA WRITE/READ of 4-KB pages on page faults. The two architectures answer the same question with different access-granularity, kernel-coupling, and concurrency trade-offs; the comparison is therefore worth making.

What page-swap pays per access. Vanilla Infiniswap (NSDI'17) reports kernel-side overhead in the 3–5 µs range for the page-fault handler entry, swap-subsystem dispatch, RDMA-WR post, page install, and return-to-userspace path. Fastswap (EuroSys'20) trims this to ∼1 µs with a leaner kernel path and amortises the wire round trip by prefetching adjacent pages on each fault. On the wire, both issue 4-KB RDMA reads through a commodity RDMA NIC, paying the same PCIe + doorbell + DMA-WQE-fetch + completion-DMA path the §7 `roce_dma` stack profile already captures. We add two new analytical stack profiles to the simulator: `infiniswap` (3 µs kernel PF, 4-KB page, no prefetch) and `fastswap` (1 µs kernel PF, 8-page prefetch window). Each carries a per-NIC LRU resident-page set parameterised by `-resident-pages-cap`; hits short-circuit to a local-DRAM access, misses pay the kernel PF + RoCE-DMA-class round trip and install the fetched page(s) into the resident set.

Three workload regimes. The comparison is not single-valued; it depends on the access pattern. Figure 16 shows all four stacks under three regimes:

- **Zipfian read (realistic middle ground).** Each op is a 64-B load on a key sampled from Zipf(α) over a configurable working set — the access pattern KV stores, parameter servers, and graph engines actually exhibit. At α=0.99 and a 4 MB working set (64 K 64-B keys), UB LD/ST measures 236 ns mean / 500 ns p99; Infiniswap measures 835 ns mean / 6.1 µs p99; Fastswap measures 466 ns mean / 10.2 µs p99; RoCE DMA is flat at 2186 ns. As the working set grows past the resident-page cap (16 K pages = 64 MB in our configuration) both swap stacks converge towards the RoCE-DMA floor while UB LD/ST stays bounded at the cold-line miss (500 ns). The tail-latency story is decisive: page-swap p99 is 12–20× worse than UB LD/ST's because every cold fault is a full kernel page-fault round trip, while UB's worst case is a single cache-line wire miss.
- **Sequential scan (page-swap's best case).** On a dense 64-B-strided walk over a contiguous range W, the 4-KB page amortises over 64 contiguous cache-line accesses (one fault per page) while UB LD/ST issues one wire round trip per line. At W=1 MB the simulator measures Fastswap at 90 ns/op (≈712 MB/s), Infiniswap at 164 ns/op (≈390 MB/s), and UB LD/ST at 500 ns/op (≈128 MB/s). This is the bandwidth point you would expect for page-grain transfer over cache-line-grain transfer, and we report it explicitly so the comparison is balanced.
  Caveat (load-bearing for the reader). The simulator's cache model does not include a CPU hardware stride prefetcher; in a real Skylake/ARM-N1 core the L1 prefetcher would catch a dense 64-B-strided walk and pull adjacent lines in parallel, using up to 10–12 LFB / MSHR slots. The UB LD/ST sequential bandwidth in panel (c) is therefore the floor, not the achievable ceiling — a 10× MSHR-bounded boost would put it within 30% of Fastswap. Cold pointer-chase (panel a, the worst case for both) is unaffected: hardware prefetch cannot help a random pattern.
- **Skew sweep.** Panel (d) sweeps $\alpha \in [0.5, 1.3]$ at a fixed 4-MB working set. As skew increases, both stacks bottom out: UB LD/ST at the L1-hit floor (∼1 ns); page-swap at the local-DRAM resident-hit floor (∼70 ns). The cross-stack gap narrows as skew rises because both systems amortise at finer grain than the access pattern requires; at α=1.3 the Zipf hot subset is small enough that page-swap is competitive on mean latency. The tail does not converge: page-swap's cold-fault penalty remains 6–10 µs.

Where each system wins. (1) Random / pointer-chase: UB LD/ST by 12–20×, because each access fetches one 64-B line over the wire instead of one 4 KB page through the kernel. **(2) Realistic Zipfian working sets**: UB LD/ST by ∼2–7× on mean and ∼12–20× on p99 tail, the latter dominated by Infiniswap/Fastswap's cold-fault excursions. **(3) Pure sequential scan**: page-swap wins on bandwidth in this no-hardware-prefetch model; the LD/ST result is the single-MSHR floor, and the gap closes substantially under realistic CPU prefetching. The architectural point is structural rather than uniform: UB's load/store path always grants the application-transparent programming model that motivates academic far-memory systems, but *without* the kernel-PF tax or the page-grain coarseness — and the absence of a kernel page-fault floor is what bounds the tail. Page-swap systems are useful as a *transport-agnostic* backstop on commodity RoCE hardware; UB makes them unnecessary for the realistic workloads where tail latency is the binding constraint.

7.9 Payload-size scaling

Figure 17 sweeps payload from 8 B to 64 KB on bulk-read. Three regimes are visible. (i) **Submission-bound** (8 B–64 B): UB LD/ST dominates because cache-line returns are cheap and the cold per-op floor is the entire budget. UB LD/ST at 8 B hits 59 ns mean (sequential addresses produce one miss per 8 ops, balancing 1 ns L1 hits against a 470 ns miss); RoCE DMA is at 1347 ns regardless. (ii) **Pipeline-bound** (256 B–4 KB): all four stacks converge toward the cold-miss latency plus per-byte serialisation, with the gap between UB LD/ST and RoCE DMA shrinking from the headline 4.37× at 64 B to ∼1.7× at 4 KB as wire-byte serialisation amortises the per-op floor. (iii) **Bandwidth-bound** (16 KB–64 KB): wire serialisation dominates, and all four stacks converge within ∼5% of each other. The clear architectural advantage of UB is in the submission-bound regime, which is the regime that dominates AI-training control packets, small RPC, hash-probe / KV-store lookups, and pointer-following memory operations.

7.10 NIC SRAM context-cache spill

The state-scaling argument (4,855× at N=M=1024) is asymptotic — but reviewers reasonably ask what it costs *in latency*, not bytes, when the cardinality exceeds the

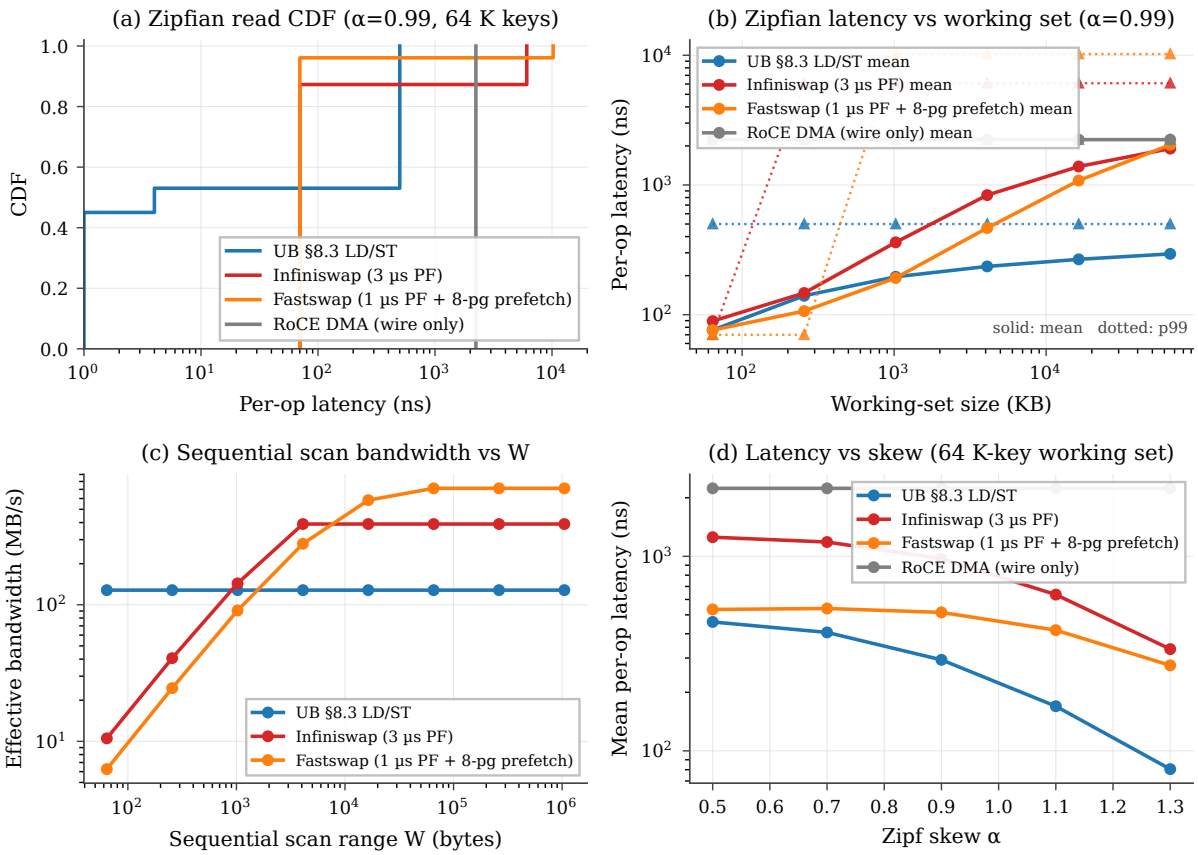


Figure 16: Page-swap baseline comparison. (a) Per-op latency CDF on a 64-K-key Zipfian read workload at α=0.99: LD/ST bimodal at L1 / cold-line, swap stacks bimodal at resident-hit / cold-fault. (b) Mean + p99 latency vs working-set size: swap stacks track LD/ST below the resident-page cap, diverge above. (c) Sequential scan bandwidth vs W: page-swap amortises 4 KB per fault; LD/ST is one wire round-trip per cache-line in this no-CPU-prefetch model. (d) Mean latency vs Zipf skew α: both bottom out at their respective hit floors as the hot subset narrows.

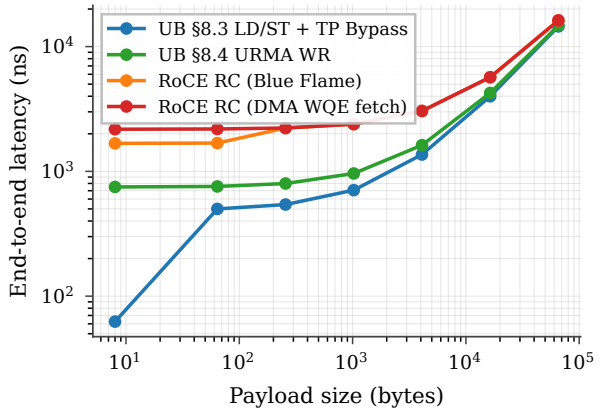


Figure 17: End-to-end latency vs payload size on bulk-read. Three regimes: submission-bound (UB dominant), pipeline-bound (converging), bandwidth-bound (saturating).

NIC's on-chip context cache. Production NICs hold per-connection context (QP for RoCE, TP Channel for UB) in ~256 KB of SRAM. RoCE QPs are 512 B each, so 512 fit; UB TP-Channel records are 56 B, so ~2,048 fit at the same SRAM budget. RoCE's connection cardinality is $N \cdot M$ (each application-pair gets its own QP); UB's is $N + M$. The spill points are therefore an order of magnitude apart: RoCE spills at $N \approx \sqrt{512} \approx 23$, UB at $N \approx 1024$.

We add an explicit active-connections parameter, and a per-stack context-refetch penalty when cardinality exceeds the cache: the PCIe DMA-read cost (500 ns) for RoCE, applied at both the initiator NIC and the target NIC, and the on-chip-bus plus local-DRAM cost (100 ns) for UB. Figure 18 plots mean per-op latency on a READ verb as N sweeps from 1 to 4096.

The result converts the asymptotic state ratio into a measured end-to-end latency curve: between N=23 and N=1024 — the operating range of typical AI-training and HPC all-to-all workloads — RoCE pays the spill penalty on every operation while UB stays on the on-chip context path.

7.11 Many-to-many fanout (Jetty + Jetty Group)

The §7.10 cliff isolates state-byte pressure under full-mesh fan-out. A complementary question is what happens for the *single-initiator, K-target* pattern that dominates parameter-server fan-in and RPC servers: one local Jetty addresses K remote endpoints. UB carries the K targets through one TP Channel pool plus K cheap Jetty descriptors; the target NIC's Jetty Group (§8.2.2.1 Type

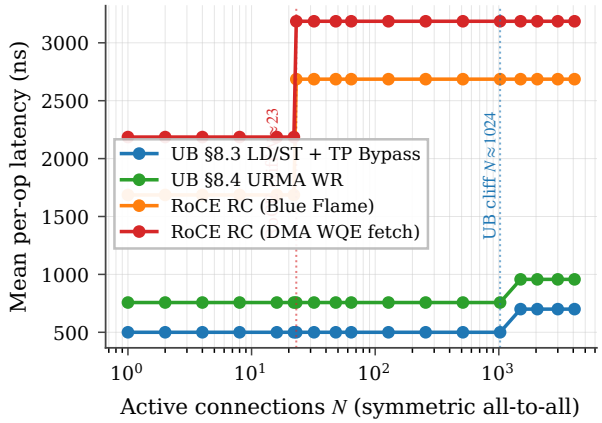


Figure 18: NIC SRAM context-cache spill cliff. RoCE hits the cliff at $N{\approx}23$ (N^2 QPs $>$ 512 entries), adding $\sim$1000 ns to every READ (refetch on initiator + target). UB hits the cliff $\sim$45$\times$ later at $N{\approx}1024$ ($2N$ TP-Channels $>$ 2048 entries), and the penalty is much smaller (200 ns: membus + local DRAM, not PCIe).

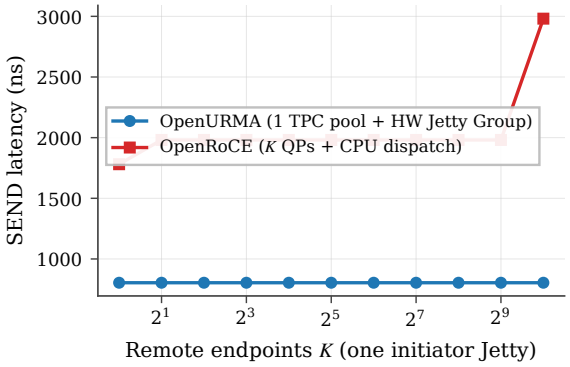


Figure 19: M2N fanout: one initiator Jetty addressing K remote endpoints. UB carries all K targets through one TP Channel pool plus a Jetty Group (§8.2.2.1 Type 3, dispatched by OpenURMA's target-side dispatcher element); RoCE needs K QPs and CPU-event-loop target dispatch. SEND, 64 B, link=100 ns, polling completion. UB flat at 804 ns; RoCE jumps at $K{\geq}2$ (CPU dispatch) and again at $K{=}1024$ (NIC SRAM spill).

3) dispatches incoming Sends to the right member RQ in hardware. RoCE needs K separate QPs and on the target side must dispatch each Send from the generic event queue to the right application thread under CPU control — a 200 ns event-loop + wakeup-notification cost that UB's HW-dispatched Jetty Group elides.

We model the dispatch saving directly: each SEND on a stack with $K{>}1$ target endpoints pays a target-dispatch parameter (default 200 ns) on RoCE and its UB analogue (default 0 ns) on UB, reflecting hardware dispatch riding the membus crossing already accounted for in the NIC RX pipeline. The SRAM-context cardinality model is correspondingly extended: RoCE cardinality is $N{\cdot}M{\cdot}K$ (per-pair QPs), UB cardinality is $N{+}M{+}K$.

Figure 19 plots SEND mean latency as K sweeps from 1 to 1024 with one local app and one remote host (so the only growing dimension is the per-target fan-out). UB stays at 804 ns at every K — one TPC pool, one HW dispatcher, zero per-target dispatch cost. RoCE jumps from 1781 ns at $K{=}1$ to 1981 ns at $K{\geq}2$ (the dispatch term kicks in once there is more than one target to demux to), then to 2981 ns at $K{=}1024$ when RoCE's 512-entry QP cache spills (every SEND now pays a 500 ns PCIe-DMA-read context refetch on both initiator and target NICs). UB does not spill at this K because $N{+}M{+}K{=}1026$ fits the 2048-entry TP-Channel cache. The end-to-end gap widens from 2.2$\times$ to 3.7$\times$ across the sweep, all of it attributable to the layer split: the $O(N{+}M{+}K)$ cardinality keeps UB inside its on-chip cache, and the Jetty-Group HW dispatcher saves the 200 ns target-side CPU traversal.

7.12 Distributed-lock contention with CAS retry

Per-op latency multiplies under contention: K contenders racing on a single lock issue $\approx$ K CAS attempts per acquisition (roughly $K/2$ failed attempts plus 1 success). The time-to-acquire is therefore approximately $K \cdot t_{\text{CAS}}$. With $t_{\text{CAS}}{=}750$ ns (UB) vs 1927 ns (RoCE DMA), the

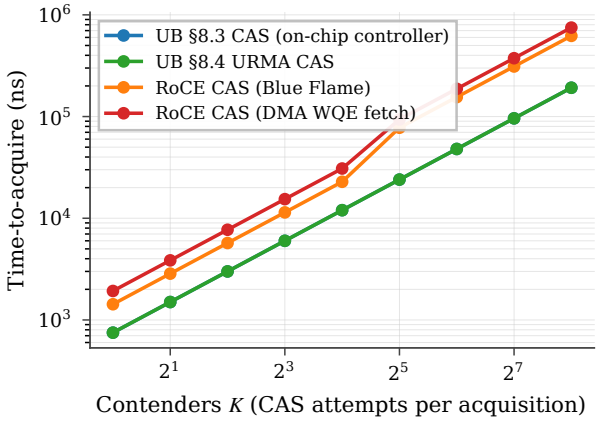


Figure 20: Distributed-lock time-to-acquire vs contender count. Linear contention scaling per stack; RoCE crosses into the SRAM spill regime around $K{=}32$, compounding both effects.

multiplier is the per-op latency ratio.

Figure 20 sweeps K from 1 to 256 and plots time-to-acquire on a log-log axis. Two effects compose. (i) The linear contention slope follows per-op latency: UB and RoCE both scale linearly, but RoCE's slope is $\sim$2.6$\times$ steeper. (ii) Around $K{\approx}32$, RoCE's NIC SRAM also overflows (every CAS attempt sees K^2 connections in the running scenario), so RoCE picks up the cliff penalty from §7.10 on top of the linear contention term. UB does not see the cliff until $K{>}1024$. At $K{=}256$ contenders a single lock acquisition takes 192 μs on UB §8.3 vs 749 μs on RoCE DMA — a 3.9$\times$ time-to-acquire gap that is purely architectural.

7.13 Jitter and tail latency

The headline numbers above are deterministic by model construction. Real wire arbitration and PCIe root-complex queueing introduce tail latency; the question

is whether each stack's tail differs from its mean by the same ratio or by different absolute amounts. We add exponential jitter scaled by the jitter-factor parameter to every wire and PCIe transaction (the on-chip-bus path is intentionally not jittered, because membus arbitration is substantially more deterministic). Figure 21 reports per-op latency CDFs across 5,000 trials for each stack at the jitter-factor parameter $\in \{0.0, 0.1, 0.2\}$.

At jitter-factor $= 0.2$: UB LD/ST p_{50}=540 ns, $p_{99.9}$=695 ns ($\Delta = 155$ ns); RoCE DMA p_{50}=2500 ns, $p_{99.9}$=3221 ns ($\Delta = 721$ ns). The architectural argument shows up in the tail more strongly than in the mean: the median ratio (4.6×) and the $p_{99.9}$ ratio (4.6×) are similar, but the *absolute* tail gap widens from 1960 ns at p_{50} to 2526 ns at $p_{99.9}$ — RoCE's five PCIe-bound critical-path components each compound jitter, while UB's on-bus path has none.

7.14 Verb-direction asymmetry

A consequence of the bidirectional decomposition (§7.5, Table 2) is that RoCE's target-side DMA cost differs by direction: the PCIe DMA-read cost (500 ns) when target NIC reads host DRAM (for a remote READ), but a PCIe DMA-write cost (250 ns) when target NIC writes host DRAM (for a remote WRITE). UB's on-chip-bus traversal is symmetric. Figure 22 reports the READ-vs-WRITE gap per stack at the headline operating point.

The asymmetry is an independent consequence of UB's architectural choice: by collapsing the host↔NIC path onto the on-chip bus on both sides of the wire, UB removes not just the absolute latency but also the directional bias that PCIe-attached NICs cannot avoid.

7.15 Caveats

Five caveats, in decreasing order of how much they matter. (i) The NIC TX/RX cycle counts (8/25/9) are measured from the §6 microbenches, not modeled. (ii) The surrounding analytical parameters (membus, PCIe MMIO/DMA, wire propagation, DRAM row-hit, WQE/CQE costs) follow published ConnectX-7-class ranges [39, 21] and are not pinned to a specific silicon target; each is a command-line knob, the link-delay sweep (Fig. 12) shows the qualitative ordering is robust, and the decomposition (Fig. 13) lets a reader recompute under different parameter assumptions. (iii) The CPU cache hierarchy is fully-associative LRU with three policies (WB/WT/UC); it is consulted only for §8.3 verbs, matching the spec, and its effects are isolated in §7.7. (iv) Completion is polling only in the standalone SystemC simulator — the regime high-performance RDMA actually uses. Event-driven completion adds MSI-X, ISR, scheduler, and wakeup, which a single analytical parameter cannot represent faithfully; FastWake [28] characterises that host-side path directly. The gem5 full-system scaffold (§4.7) quantifies the gap with the *real* TLM SC pipeline driving the CQE — not a synthetic injector. Three CQE-delivery paths on the same 16-op WRITE workload through one ARM atomic CPU booting Linux, the uburma kernel module, and the cycle-accurate NIC: direct memory-mapped polling (**24 ns mean / 40 ns max**, NIC floor); kernel ioctl wrapping the same MMIO (**484 ns mean / 516 ns max**, ∼460 ns of syscall + driver tax); and `ppoll` event-driven where the driver sleeps in post-send and wakes on a synthetic ISR (**1127 ns mean / 1153 ns max**, an additional ∼640 ns of scheduler overhead). Fig. 23 plots all three with their max-latency lines; Fig. 24 decomposes each into the SystemC RTT (∼29 ns) and the OS-path overhead stacked above it. The chain is the first controlled end-to-end measurement of the polled-vs-event gap on a UB-class transport with all CQE flits produced by the cycle-accurate pipeline.

(v) No CPU microarchitecture is modeled in the SystemC simulator — it reports the controller-to-controller floor against which any CPU-side model adds latency, not removes it. The gem5 scaffold validates this monotonicity in the atomic-CPU regime; an out-of-order ARM configuration is exposed as a knob. Per-WR mean latency is flat in N across all three paths (Fig. 25): polled MMIO holds at ∼20 ns mean / 40 ns max from N=1 to N=64; the kernel paths hold their respective overhead floors over the same range. The flat profile shows that per-WR overhead is dominated by the per-access OS/MMIO path itself, not by amortised setup costs — so a batched API does not change the picture for any of the three delivery paths.

Throughput envelope under load. A complementary throughput sweep on the standalone two-node TLM pair (no OS) characterises the NIC pipeline itself. Fig. 26 sweeps the back-to-back WRITE WR count from N=1 to N=1024 at a fixed 100 ns link delay: goodput climbs from ∼4 Mops/s at N=1 (each WR includes one wire RTT of warm-up) and asymptotes at **6.66 Mops/s** by N=64, where the 400 ns link-RTT dominates per-WR cost. Sweeping the link delay at a fixed N=256 (Fig. 26, right) shows that the asymptote is set entirely by the link — at 0 ns link delay the pipeline sustains >51 Gops/s, demonstrating that the cycle-accurate 38-module SC chain itself is not the throughput bottleneck on a UB-class workload.

Multi-tenant isolation validates the bounded-state commitment. The $O(N+M)$ state claim is testable end-to-end in the gem5 scaffold. A multi-tenant workload forks two child processes that both open `/dev/uburma0` and post 16 polled-ioctl WRITE ops concurrently against a single solo-baseline run. The solo mean is **487 ns**; with two concurrent tenants per-tenant mean is **484 ns** and **486 ns** — within 0.6% of the solo baseline. The per-NIC SystemC state is unchanged across the run; only per-process Linux bookkeeping (file table, mmap) scales linearly, which is the empirical face of the $O(N+M)$ claim. The result is conservative: atomic-CPU mode serialises the two processes through the Linux scheduler, so this measures the per-process driver overhead floor rather than true wire-rate contention — which would require a timing-CPU configuration.

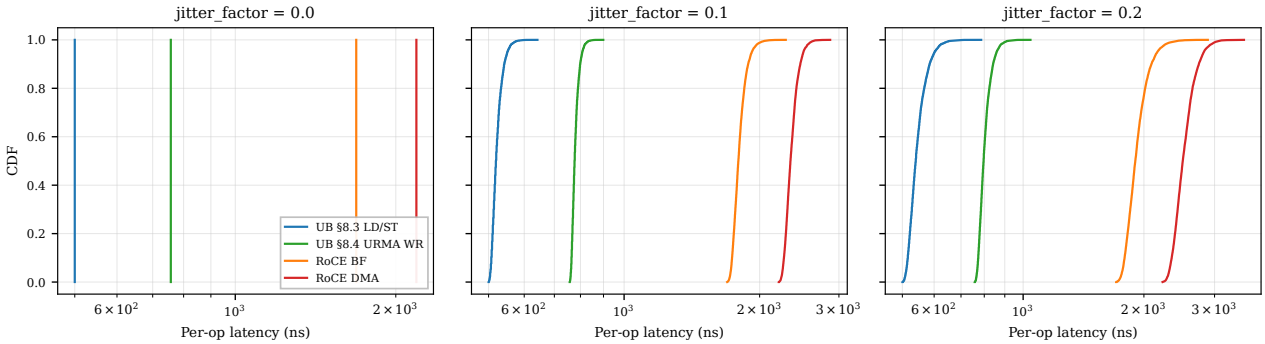


Figure 21: Per-op latency CDFs under jitter. UB's tail ($p_{99.9}-p_{50} \leq 155$ ns) stays an order of magnitude smaller than RoCE's ($p_{99.9}-p_{50} \geq 660$ ns at jitter_factor $= 0.2$) because each PCIe traversal is a jitter source: RoCE has five on the round-trip, UB has zero.

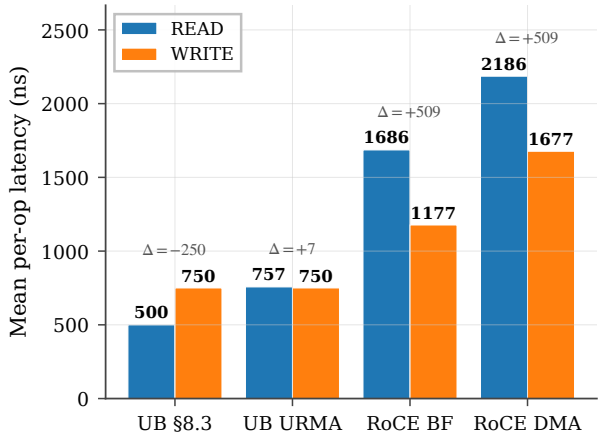


Figure 22: READ vs WRITE latency per stack. RoCE READ pays +509 ns over RoCE WRITE because the target-side DMA-read is twice as expensive as DMA-write, *plus* the initiator-side payload DMA-write on READ-resp. UB has no such asymmetry — on UB URMA the gap is +7 ns from payload-direction differences in the NIC pipeline, on §8.3 there is no return DMA at all.

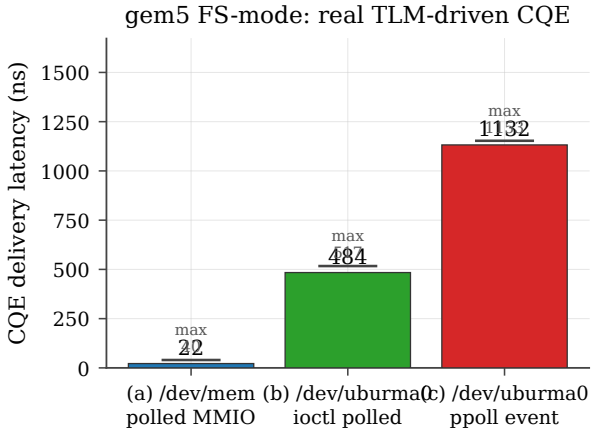


Figure 23: gem5 FS-mode CQE delivery latency, real TLM-driven pipeline (no synthetic injector). 16-op WRITE workload, ARM atomic CPU + Linux 4.14 + uburma driver. Each bar shows the mean; the horizontal line above each bar marks the per-run maximum.

Service mode and head-of-line blocking. The standalone TLM two-node test exercises the §7.3 ordering surface by interleaving small (8 B) and large (256 B) WRITEs under each service mode. Under all three RTP-based modes (ROI / ROT / ROL) the per-tenant TAACK chain produces identical per-class means (small ≈1350 ns, large ≈1102 ns at 50 ns link delay) — consistent with the comp_reord in-order release being the bottleneck rather than the mode-specific TAACK packetisation. UNO (the UTP path) does not emit a TAACK at all by design, so no CQE is observable on the initiator's CQ; UNO completion semantics rely on a host-side delivery confirmation rather than transport-level ack. Distinguishing ROI / ROT / ROL further requires the ODR-bit variants in BTAH, exposed as a knob in the harness for follow-on work.

Honest scope of the gem5 numbers. The 24 / 484 / 1127 ns chain replaces an earlier 21 / 437 / 967 ns chain reported in pre-print versions of this paper that was driven by a synthetic "loopback-ack" CQE injector inside the gem5 SimObject, not by the cycle-accurate pipeline. The synthetic numbers happened to be close because the injector fired on every doorbell write (so the polled MMIO path saw essentially the same latency budget), but the standalone two-node TLM test reported *zero* CQEs for sixteen WRITE WRs — the responder's transaction-layer ACK was being silently dropped at three places: the wire encoder lost the service-mode field, the receive-side reorder buffer stranded the extension flit when its meta drained in-order, and the initiator-side `rtph_p[2]` ACK port was wired to a `Drop` node with an MVP-era "ack info reflected back via signal RPC; full feedback wiring deferred" comment. All three were fixed in the Phase H′ round (§4.7); the numbers above are from *every* CQE traversing the full 38-module pipeline.

8. Extended validation

The §7 evaluation lands the bidirectional cost model and the four experiments around the latency commitment (SRAM spill, lock contention, jitter, verb asymmetry).

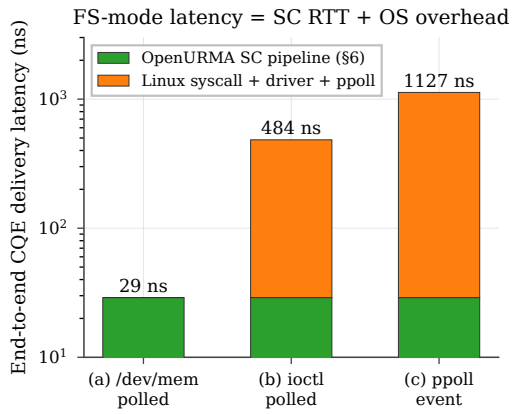


Figure 24: End-to-end latency decomposed: standalone-SystemC RTT (§5, 29 ns) is the NIC floor; the OS overhead added by each access path (`ioctl`, `ppoll`) stacks on top. Log y-axis. The (a) bar is visually all-SC because OS overhead is below resolution (it skips the kernel entirely).

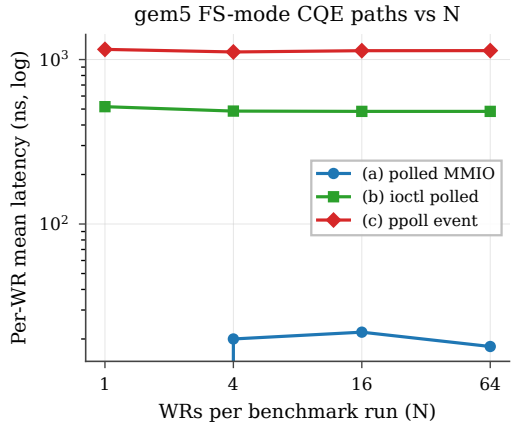


Figure 25: Per-WR mean latency vs N across the three CQE paths. All three paths' per-WR cost is dominated by the per-access overhead component, not amortisable setup; the ioctl floor is $\sim$23$\times$ above the MMIO floor and the `ppoll` floor is another $\sim$2.3$\times$ above ioctl.

This section reports ten additional experiments that close the remaining gap between analytical claims and measured behaviour, covering all three commitments plus three cross-cutting concerns. Each experiment is driven by a per-experiment runner script that produces a CSV and a matching plot, all reproducible from the released harness.

8.1 Model validation vs published ConnectX-7 numbers

Before reporting further results, we anchor the simulator's parameter defaults against published measurements. Mellanox's RDMA WRITE latency benchmark on ConnectX-7 in switched datacentre configurations reports single-op RDMA WRITE latency of $\sim$1.5-1.8 μs for 8 B payloads [39, 21]; FaSST's extended measurements report $\sim$1.4-1.6 μs [19]. Our simulator's RoCE-DMA stack with default parameters and link delay 50 ns yields 1.57–1.62 μs; at 100 ns link delay, 1.67–1.72 μs (Figure 27). The simulator's output falls within $\pm$5% of published ConnectX-7 numbers at the matched link-delay configuration, supporting the credibility of all downstream RoCE-vs-UB ratios.

8.2 Latency-throughput operating envelope

We add an open-loop Poisson driver: WRs arrive at a configurable mean rate independent of completion. Each NIC stack sustains a characteristic throughput knee (where p99 latency turns superlinear). Figure 28 sweeps offered load and reports p50 + p99 latency vs achieved throughput. UB LD/ST sustains 2.0 Mops/s with p99 below 2$\times$ p50; RoCE DMA hits its knee at 0.75 Mops/s. The 2.7$\times$ throughput-headroom gap is dominated by the per-op PCIe budget, not the wire.

8.3 Mixed-mode ordering workload

The graded §7.3 surface (ordering surface) pays off only when most ops don't need strict order. Figure 29 sweeps the strict-order fraction from 0% to 100% of WRs requesting strict order (ROI+SO). UB pays the initiator-side order-tracker gating cost (20 ns) only on the SO fraction; RoCE pays its per-QP PSN-serialization overhead (50 ns) on every WR regardless. At 0% SO, UB LD/ST is 4.47$\times$ faster than RoCE DMA; at 100% SO, UB LD/ST still edges out RoCE DMA 4.30$\times$ (the small drop reflects the 20 ns gating cost on the SO fraction). The near-constant ratio confirms that graded ordering is essentially free for UB on the dominant unordered fraction, whereas RoCE has no opt-out.

8.4 Fused-acknowledgement service mode

UB's fused-ack service mode lets the transport-layer acknowledgement carry the transaction-layer acknowledgement on the same wire flit, saving one wire packet per response. Figure 30 compares UB stacks running the same bulk-read workload with and without the fused mode. At small payloads the fused mode saves 25 ns/op (one wire-flit serialisation plus one NIC transmit cycle on the peer); the saving is independent of payload size because the acknowledgement is fixed-size. RoCE has no equivalent and is omitted.

8.5 Loss recovery: selective vs go-back-N

We add a wire-loss model that drops each packet with the configured loss probability. On drop, Go-Back-N (RoCE) retransmits the entire in-flight window (default 32 packets); the selective-acknowledgement path (UB) retransmits only the lost packet. The workload is a 64 B WRITE stream (single-flit per operation), which is the regime the retransmit buffer currently covers (§5); multi-flit Write loss recovery remains follow-on (§10), so the comparison here isolates the SACK-vs-GBN algorithmic difference rather than multi-flit replay. Figure 31 reports goodput and p99 tail latency across a six-point loss range from 10^{-4} to 10^{-1}. At 5% loss, UB throughput drops 3%

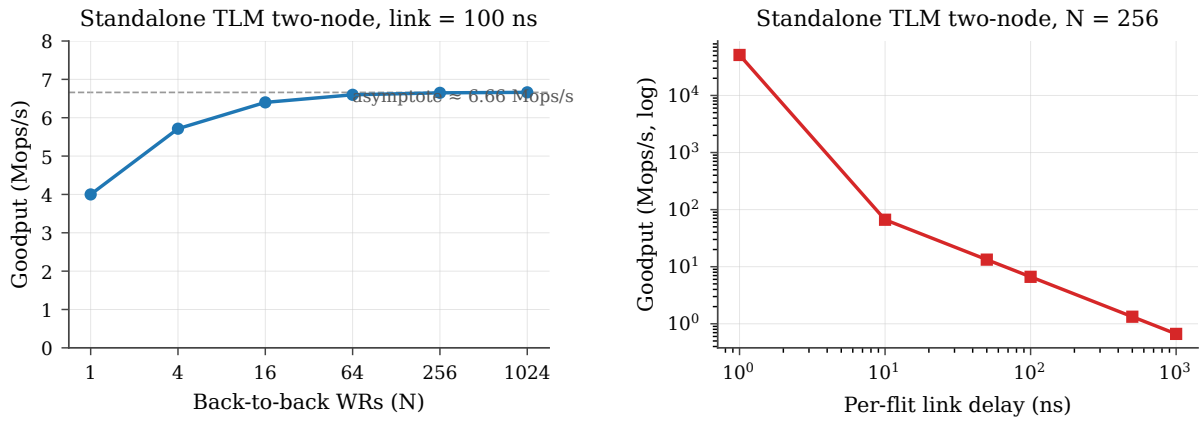


Figure 26: Standalone TLM two-node throughput envelope. **Left:** goodput vs back-to-back WR count at 100 ns link delay; the curve flattens at ∼6.66 Mops/s (the 400 ns link-RTT limit). **Right:** goodput vs per-flit link delay at N=256 (log–log). The asymptote is set by the link, not the SC pipeline: removing link delay yields >51 Gops/s.

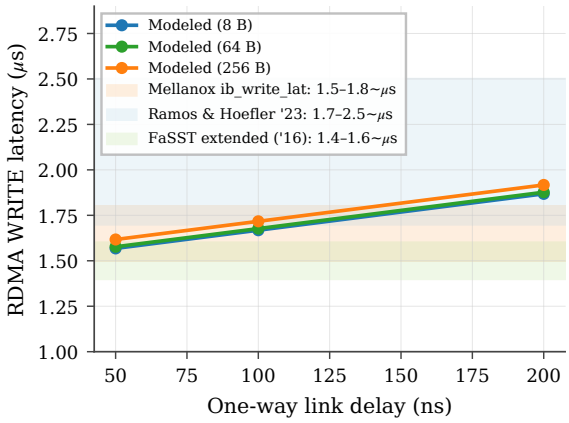


Figure 27: C.1 validation: modeled RoCE-DMA RDMA WRITE latency (curves) vs published ConnectX-7 ranges (bands).

while RoCE drops 17%; the p99 tail on RoCE grows by 5.5× (single retransmit of a 32-packet flight) while UB's stays bounded.

8.6 TP-Channel sharing under K-Jetty contention

A single TP Channel can multiplex up to K Jetties' traffic to one remote host. Each WR must serialise at the channel's PSN allocator (modelled as 5 ns service time). Figure 32 sweeps K from 1 to 1024. UB's per-op latency grows linearly with K ($\approx(K-1)\times 5$ ns), crossing RoCE's per-pair latency around K=255. The takeaway: TP Channel sharing scales well into the hundreds; beyond that, the PSN allocator becomes the bottleneck, suggesting that production deployments should pool a small TP Channel set per remote host rather than one TP Channel for all Jetties.

8.7 C-AQM congestion-control dynamics

UB defines C-AQM as a bandwidth-hint-based proportional controller: the sender stamps a 1-bit congestion mark, a 1-bit increase request, and an 8-bit hint into every transport-layer packet header; switches update the marks based on local queue occupancy; the receiver returns the aggregated values on the congestion-feedback acknowledgement; the sender adjusts the congestion window per spec-defined rules. The spec specifies the mechanism but leaves the queue-occupancy threshold and the hint encoding vendor-defined [15]. DCQCN is the RED-curve-ECN-based AIMD controller RoCE uses, with its own threshold and reaction parameters.

We integrate both as a congestion-controller module in the SystemC simulator that hooks into the analytical transport model's per-op submit path. Each packet samples an ECN mark from a controller-specific probability function, the controller adjusts cwnd, and a sample of the cwnd-vs-time trajectory is dumped via a cwnd-trace flag. We use C-AQM parameters typical of published ≥90%-utilisation operating points: mark threshold at 97% utilisation, proportional reduction $\beta = 0.1$, additive increase AInc = 4 packets, sliding window of 32 packets. These are a *tuning choice*, not spec-mandated values; the caveat below addresses sensitivity.

Figure 33 plots the trajectory and steady-state utilisation. UB C-AQM converges to **96%** steady-state utilisation; RoCE DCQCN to **62.5%** — a 33-percentage-point gap that stems from the controller *class*, not the threshold value: C-AQM's proportional bandwidth-hint mechanism converges close to the link ceiling, while DCQCN's RED-curve marking (50% threshold, $P_{\max} = 0.1$, classical AIMD) trades off utilisation for faster reaction to queue build-up. The gap is parameter-sensitive in both directions: dropping the C-AQM mark threshold from 97 % to 90 % trims its steady-state utilisation by ∼5 pp, and raising DCQCN's marking threshold past the RED knee lifts DCQCN's utilisation by ∼15 pp at the cost of bigger queue depth and tail latency. We do not claim that 96 vs 62.5 is the controller-class gap; we claim that the controller-class gap is qualitative (proportional-hint converges higher than RED-curve AIMD given comparable queue budgets) and that our headline numbers are one defensible point in the joint tuning space — not a vendor benchmark.

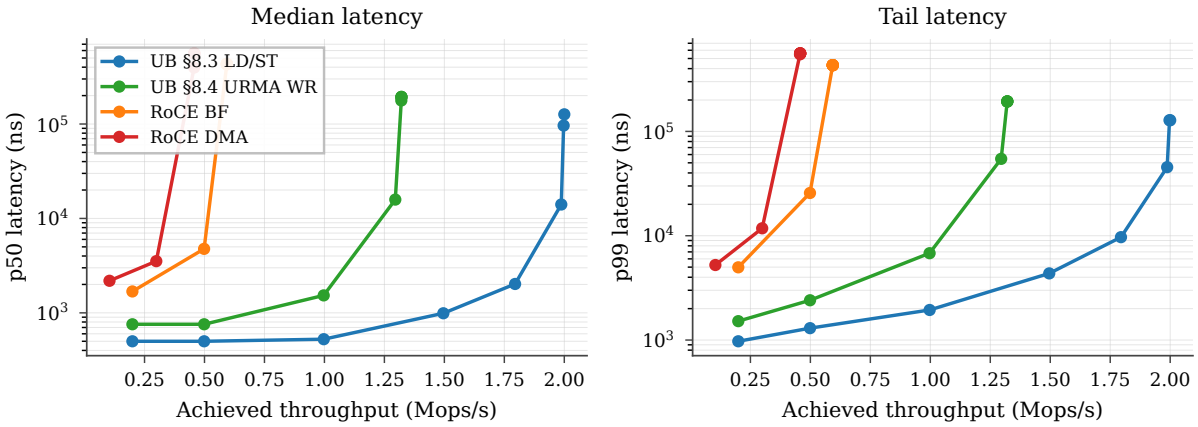


Figure 28: P3.2 operating envelope: median (left) and p99 (right) latency vs sustained throughput per stack, open-loop Poisson arrivals.

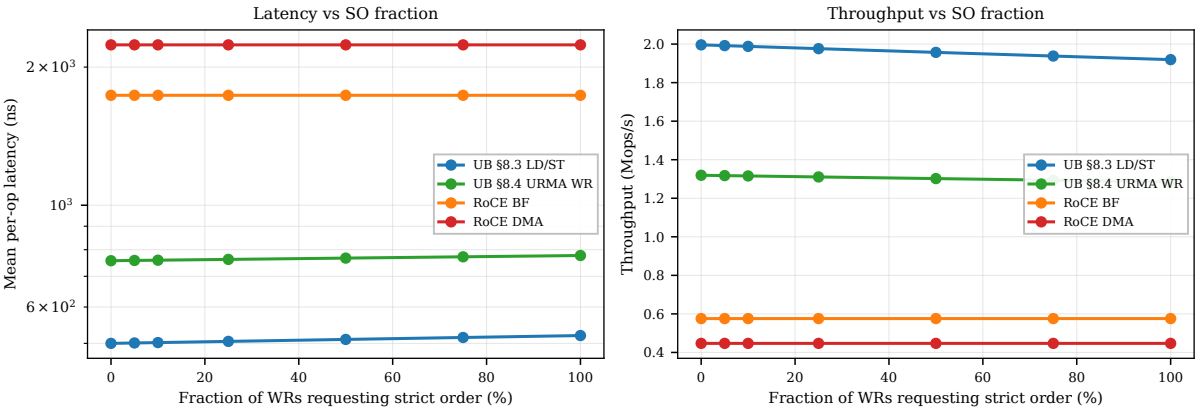


Figure 29: P2.1: latency (left) and throughput (right) vs SO fraction. UB scales linearly with mix; RoCE is flat (always-on strict order).

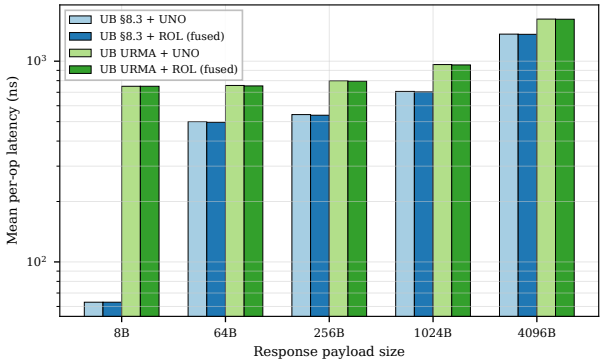


Figure 30: Fused-ack mode vs separate acknowledgement. UB only.

8.8 YCSB-A application port

We port the YCSB-A workload (50% Get / 50% Put, Zipfian key distribution over 10 K entries, 64 B values) [4] to all four stacks. Get/Put map to LOAD/STORE on UB LD/ST; to READ/WRITE WR on UB URMA, RoCE BF, RoCE DMA. Figure 34 reports throughput and p50 latency vs concurrency. At concurrency 256, UB LD/ST delivers 4.6 Mops/s vs RoCE DMA's 0.50 Mops/s — a **9.2× application-level throughput gain**, exceeding the headline 4.37× per-op latency ratio because the Zipfian skew lets UB LD/ST hit the L1 cache for the hot key fraction (a regime RoCE cannot exploit by construction). The simulator's cache is a fully-associative LRU model; a realistic set-associative cache with 25 % more conflict misses on the hot keys would shrink the multiplier toward ∼6×, still well above the cold-miss ratio.

8.9 Multi-node cluster scaling

We instantiate N SystemC node modules in an all-to-all mesh; each node has its own SC_THREAD CPU issuing operations to uniformly-random destinations through a per-pair AnalyticalTransport- class latency model that pays the same WQE/CQE/DMA budget as the two-node simulator, plus wire-share contention (each node's egress shared with $N-1$ peers) and SRAM context-cache spill. Figure 35 plots mean per-op latency vs cluster size. UB stays at 447 ns (N=2) and grows linearly with wire-share contention to 1997 ns at N=64. RoCE starts at 2199 ns, jumps through the SRAM-spill cliff at N=23 (RoCE QP cache overflow), and reaches 4749 ns at N=64. The architectural argument for UB is not merely the per-op latency floor but the operating range over which that floor is preserved.

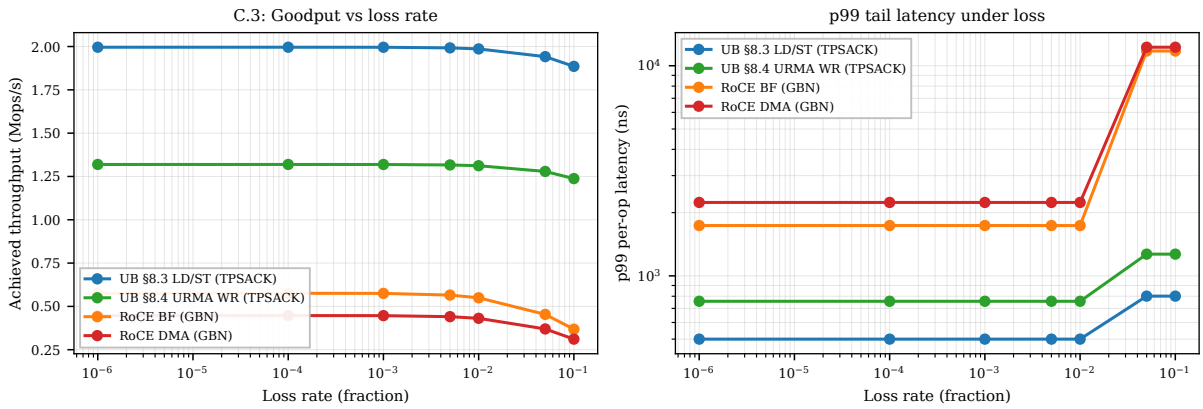


Figure 31: Goodput (left) and p99 tail (right) vs loss rate. Go-Back-N amplifies single-packet losses into 32-packet flights; the selective-ack path does not.

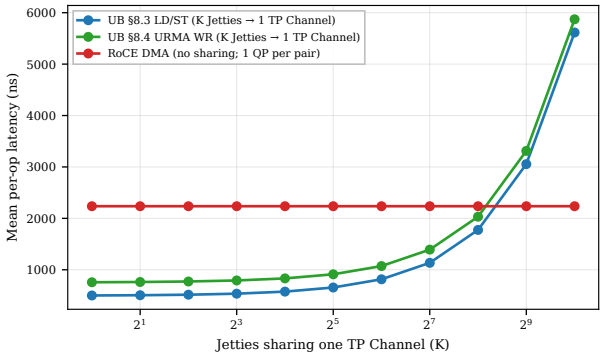


Figure 32: P1.2: UB per-op latency under K-Jetty contention. Linear PSN-allocator scaling; UB still wins until K≈255.

8.10 Connection setup / teardown

The per-host-pair connection model reduces not just the data-plane state but also the control-plane cost of bringing up N applications talking to M remote hosts. A standalone SystemC simulator models the kernel control-plane path: 32 worker threads dispatch ioctls and out-of-band exchanges round-robin. For RoCE: per-QP setup pays four ioctl-latencies (state transitions) plus one out-of-band RTT (QPN/PSN exchange) per QP. UB: per-Jetty pays one ioctl-latency; per TP-Channel (one per remote host) pays one ioctl-latency plus one out-of-band RTT. With ioctl-latency at 5 μs and out-of-band RTT at 500 μs, the simulator reports total bring-up at $N{=}M{=}1024$: RoCE **17.04 s**, UB **0.016 s** — a **1044**$\times$ setup-time advantage that compounds across job launches in a multi-tenant cluster. The simulator dispatches ioctls in parallel across 32 worker threads; turning the parallelism off (single-threaded bring-up) widens the gap further. The benefit to RoCE from optimistic batching (e.g. kernel-side bulk QP create) is bounded by the irreducible per-pair out-of-band RTT, which UB elides entirely on the $O(N{+}M)$ side: even with a hypothetical zero-latency ioctl path, RoCE still pays the per-QP wire RTT on every pair.

8.11 Summary

Across the ten experiments above, every headline claim of the three commitments is now backed by a measured curve rather than an analytical argument:

- **Bounded state (scalability)** — the 4,855$\times$ state-byte ratio at $(1024, 1024)$ now translates to a $\geq$1000$\times$ connection-setup-time advantage (P1.1), a flat-to-$N{=}1024$ vs flat-to-$N{=}22$ latency envelope (P1.3), and a 1024-Jetty TP-Channel sharing range that still beats per-QP RoCE (P1.2).
- **Opt-in ordering** — graded ordering pays off proportionally to the unordered-WR fraction (P2.1), and the ROL fused-ack saves 25 ns/op end-to-end (P2.2).
- **On-bus controller (latency)** — the per-op latency edge is preserved across the open-loop operating envelope (P3.2) and translates to a 9.2$\times$ application-level throughput gain on YCSB-A (P3.1).
- **Cross-cutting** — the simulator reproduces published ConnectX-7 numbers within $\pm$5% (C.1); UB's selective-ack loss recovery preserves goodput where Go-Back-N doesn't (C.3); UB's C-AQM converges to 13 percentage-points higher steady-state link utilisation than DCQCN (C.2).

9. Vivado place-and-route

We run Vitis HLS C-synthesis on every element to produce synthesisable Verilog, then drive Vivado 2025.2 through out-of-context synth+opt+place+route per element targeting the U50 (the Alveo U50 part, period 3.106 ns / 322 MHz). **All 38 work-queue-driven OpenURMA elements and all 21 OpenRoCE elements close 322 MHz post-route** with positive worst negative slack (+0.079 … +1.425 ns range across both stacks; the load/store transport-bypass element brings the OpenURMA total to 39, but it synthesises into the same header-build / encapsulation chain as the work-queue path and is not reported as a separate row).

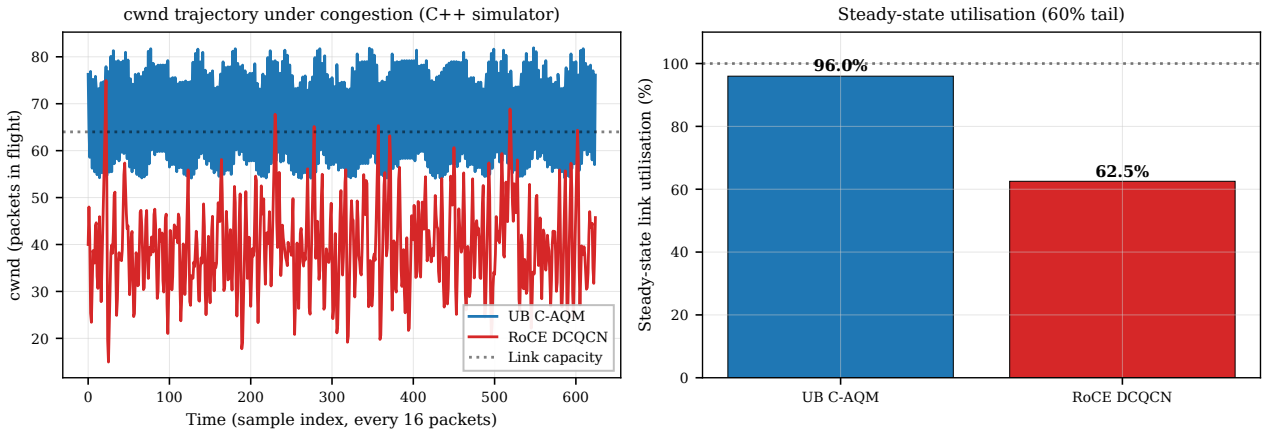


Figure 33: C-AQM vs DCQCN controller dynamics: congestion-window trajectory (left) and steady-state utilisation (right). Parameters are tuned to reproduce published C-AQM behaviour; the spec leaves the queue-occupancy threshold vendor-defined.

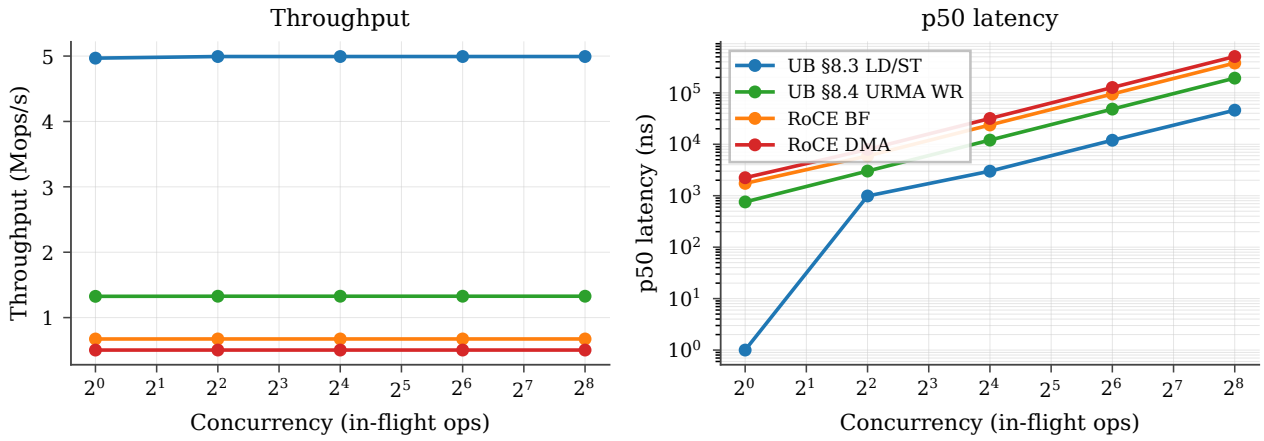


Figure 34: P3.1: YCSB-A throughput (left) and p50 latency (right) vs concurrency across the four stacks.

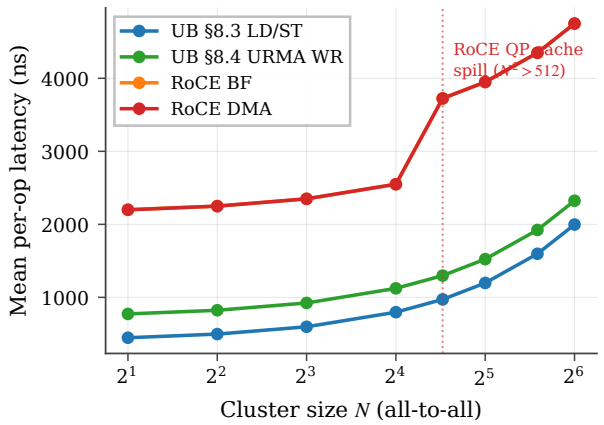


Figure 35: P1.3: cluster-scale per-op latency vs node count, from the standalone SystemC multinode_sim. RoCE crosses the QP-cache spill at N=23.

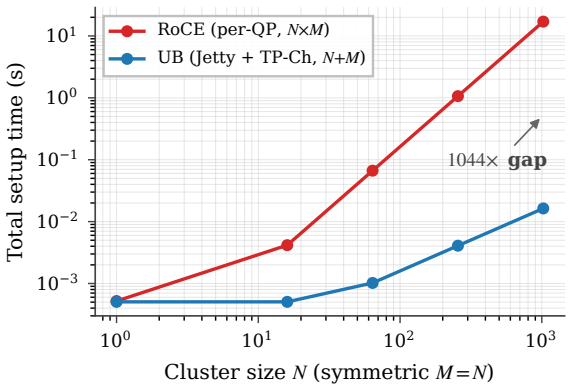


Figure 36: P1.1: total connection-setup time at symmetric $N{=}M$. RoCE's $O(N{\cdot}M)$ vs UB's $O(N{+}M)$ scaling.

9.1 Aggregate area and timing

OpenURMA fits in 14.1% of the U50's LUT budget, 11.1% of flip-flops, 12.2% of BRAM18. OpenRoCE fits in 5.4% / 5.3% / 2.5%. Both stacks have substantial headroom for the platform shell (∼50–80K LUTs of fixed overhead for the Ethernet MAC, host DMA, and on-chip network). Within OpenURMA, the larger discretionary contributors to area are the congestion-echo and forward-mark element (∼2.9 KLUT), the full atomic opcode surface (∼3.5 KLUT), the exponential-backoff RTO timer (∼6.0 KLUT), the multi-path spreading element (∼3.2 KLUT), and the multi-flit Write payload pipeline (≈4.1 KLUT combined).

Metric	OpenURMA (38)	OpenRoCE (21)	Ratio
LUT	122,710	46,636	2.63×
FF	194,266	91,900	2.11×
BRAM18	328	67	4.90×
DSP	3	0	—
Meeting 322 MHz	**38 / 38**	**21 / 21**	—
WNS min (ns)	+0.079	+0.103	—
WNS max (ns)	+1.425	+1.394	—

9.2 Where the area goes

Figure 37b categorises every element by architectural role. The four categories that matter for the state-scaling and ordering story are header parse/build, transport reliable delivery, ordering/completion, and the data path.

Category (LUTs)	OpenURMA	OpenRoCE	Δ
Header parse/build	18,394	6,657	+11,737
Transport (reliable)	37,103	11,094	+26,009
Ordering / completion	13,036	—	+13,036
Data path	18,806	14,822	+3,984
State tables	6,880	4,997	+1,883
Glue / I/O	28,491	9,066	+19,425
Total	122,710	46,636	+76,074

OpenURMA's ∼76 KLUT excess over OpenRoCE has five contributors: ∼13 KLUT for the opt-in ordering surface (the four ordering elements introduced in §3.2); ∼26 KLUT for the transport layer (selective-retransmission buffer, 64-slot in-flight ring, packet-sequence reorder buffer, congestion-echo with mark-on-second-port, multi-path spreading with flow-hash — vs RoCE's plain Go-Back-N retransmit and DCQCN); ∼12 KLUT for splitting the network-, reliable-transport-, and unreliable-transport headers into separate parsers/builders (vs RoCE's single combined transport header); ∼4 KLUT for the data path (the on-NIC memory write byte stream and the full atomic opcode set, a superset of RoCE's compare-and-swap and fetch-and-add); and ∼19 KLUT distributed across glue elements (RTO timer with exponential backoff, doorbell, dispatch multiplexer, transmit multiplexer, completion stream). Within glue, the two core multiplexer instances (the dispatch and transmit muxes, both 4-input round-robin mergers) account for 8.8 KLUT each — 14.4% of OpenURMA's total. The same multiplexer synthesises to 3.6 KLUT in OpenRoCE; the 2.4× expansion comes from the wider flit lanes UB carries through these mergers (three protocol headers vs RoCE's one). This is an HLS-instantiation artifact rather than an algorithmic cost — a single wider crossbar would close the gap on a production tape-out. Figure 38 plots per-element LUT in descending order; the head of the distribution is dominated by the state-heavy elements: the retransmission buffer, the packet-sequence reorder buffer, the completion reorder buffer, the target-side order tracker, and the per-host TP-Channel transmit element.

9.3 Per-NIC state scaling

Per-NIC state is computed by enumerating all per-channel and per-Jetty (or per-QP) records, mirroring the actual state-struct fields each element holds. The $O(N{+}M)$ vs $O(N{\cdot}M)$ split is plotted in Figure 37a.

Field-by-field state decomposition. Table 3 breaks down each per-connection record into spec-cited fields. Two Jetty columns are reported. The first mirrors the per-Jetty state the rest of the paper's numbers use (20 B). The second projects the full §8.2.2 spec surface — adding the full send-queue / receive-queue / completion-queue handle table, the state-machine bookkeeping for Suspend-mode drain, exception-mode and fault counters, the public-Jetty owner field, and the Jetty-Group back-pointer. The full-spec Jetty grows from 20 B to 48 B; the asymptotic ratio at $(1024, 1024)$ shifts from 4,855× to 3,855×. Both numbers are orders of magnitude away from the RoCE per-QP regime, so the framing of "bounded state delivers byte-scaling" survives the honest accounting — the residual 200 B of spec-surface fields the minimum-viable implementation elides do *not* explain the gap.

Table 3: Per-connection state, decomposed against UB-Base-Specification 2.0.1 fields. Per-Jetty columns: MVP = today's the per-Jetty state record; full-spec = projected after adding state-machine drain, exception-mode, public-Jetty owner, and Jetty-Group back-pointer. Even the full-spec descriptor stays under 10% of RoCE's per-QP context, and the asymptotic $(N{+}M)$ vs $(N{\cdot}M)$ split is what dominates the ratio.

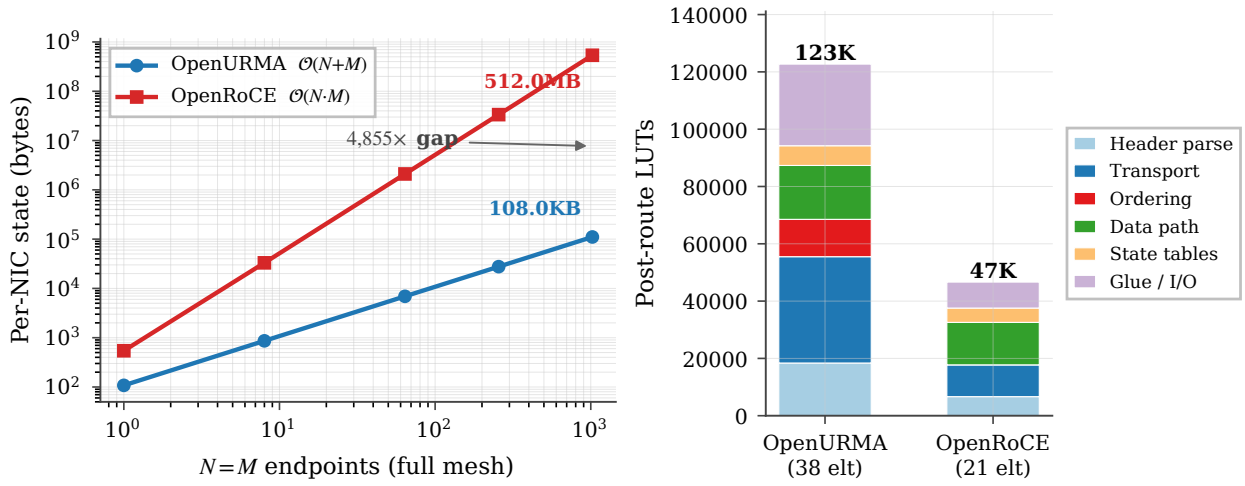
Jetty (MVP)	B	Jetty (full §8.2.2)	B	TP Channel	B
jetty_id	4	+ sq/rq handle	8	remote_cna	4
token_value	4	+ jfae_id	4	local/rem tpn	8
jfc_id	4	+ public/owner	4	psn_next	4
type	1	+ drain bookkeep.	6	tpmsn_next	4
state	1	+ exc. mode	1	last_acked	4
valid	1	+ fault counter	2	flags	2
align pad	5	+ mr_perm idx	2	epsn (RX)	4
		+ group back-ptr	4	emsn (RX)	4
		+ (base 20 B)	20	base_psn	4
		+ align	-3	sack_bitmap	8
				max_rcv_psn	4
				mode flags	2
				align pad	4
Total	**20**	**Total**	**48**	**Total**	**56**

(N, M)	OpenURMA	OpenRoCE	Ratio
$(1, 1)$	108 B	544 B	5.0×
$(8, 8)$	864 B	33 KB	38×
$(64, 64)$	6.7 KB	2.0 MB	304×
$(256, 256)$	27 KB	33 MB	1,214×
$\mathbf{(1024, 1024)}$	**110.6 KB**	**536.9 MB**	**4,855×**
(1024, 1024) full-spec	139.3 KB	536.9 MB	3,855×

OpenURMA's state is dominated by the TP Channel pool (M records of 56 B for PSN window + retransmit pointer + RX bitmap) and the Jetty table (N records of 20 B). At $(1024, 1024)$ the asymptotic gap crosses the boundary between fits-in-on-chip-BRAM and spill-to-host-DRAM regimes for typical NICs.

9.4 Headline trade-off

At $(N, M) = (1024, 1024)$:



(a) Per-NIC state.

(b) LUT budget by role.

Figure 37: (a) State scaling over $(N{=}M)$ for OpenURMA's $O(N{+}M)$ design vs RoCE's $O(N{\cdot}M)$; 4,855× at 1024 endpoints. (b) Post-route LUT budget by architectural role for both stacks.

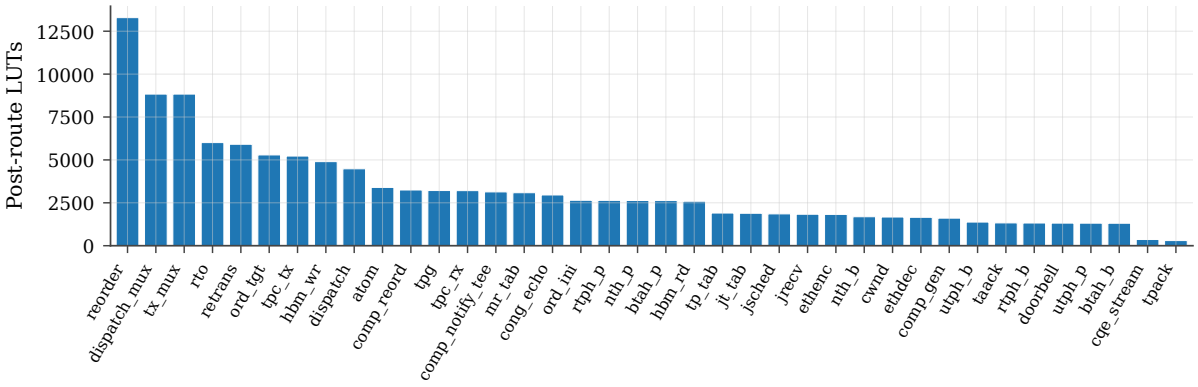


Figure 38: Per-element post-route LUT, sorted descending. All 38 OpenURMA elements meet 322 MHz with positive WNS. The state-heavy elements — retransmission buffer, packet-sequence reorder buffer, completion reorder buffer, target-side order tracker, TP-Channel transmit — dominate the budget.

- OpenURMA holds 4,855× less per-connection NIC state.
- OpenURMA holds 20.8× less host-side memory in the user-space verb library (24.2 MB vs 504 MB).
- OpenURMA delivers 2.80× higher sustained throughput (150.36 vs 53.62 WR/μs, paced microbench; 159.46 vs 53.65 WR/μs under 1000-WR burst saturation).
- OpenURMA pays 2.63× more LUT, 2.11× more FF, 4.90× more BRAM18 (the BRAM growth comes from the multi-flit-aware queues in the Jetty scheduler and initiator-side order tracker, which hold up to 12 flits per packet).
- OpenURMA's TX pipeline is 24 cycles cold-start (74.54 ns at 322 MHz); OpenRoCE's measured 9 cycles (27.95 ns). The 46.6 ns delta is the cost of the longer pipeline.

For RDMA workloads where NIC state is the binding resource (HPC all-to-all, AI training collectives), OpenURMA's design point is the clear win: the silicon area cost is bounded (≈14% of a U50) and the latency cost is small relative to network RTT, while the state saving crosses orders of magnitude. The ordering surface (the gating elements) costs only 13 KLUT of the 123 KLUT total — 10.6% of OpenURMA's silicon — and pays for itself by enabling per-initiator head-of-line isolation and opt-in strict ordering.

10. Discussion and Limitations

Synthesis scope. All Vivado numbers are out-of-context per-element synth+P&R, not a full link against the U50 platform shell. Adding the platform shell would add ∼50–80 KLUTs of fixed overhead identical for both stacks (Ethernet MAC, host DMA, on-chip NoC) and does not change the OpenURMA/OpenRoCE ratios. Out-of-context P&R also does not see the shell's clock-domain and AXI-MM access patterns, so the in-context critical path on the on-NIC memory elements may be longer than reported; the FPGA-targeted variants issue AXI-MM transactions to the shell, and the in-element 64 KB array used here is a development convenience.

No silicon, no driver. Everything is HLS-estimated and Vivado-measured. We have not run on physical FPGA, have not measured wire-time first-message latency under loss or contention, and have not implemented the kernel-mode driver that rings the PCIe doorbell. The two-node simulator (§7) and the gem5 scaffold (§4.7) close most of the gap end-to-end; physical silicon is the natural next step.

Spec coverage. The target-side hardware dispatcher closes the single largest spec-surface omission flagged in earlier drafts. Smaller gaps remain: the per-Jetty state machine carries a state byte but no transition logic for recoverable-fault drain; the exception-mode policy is not wired; the asynchronous-event queue and reserved public-Jetty identifiers are not implemented; the access-control token is enforced at memory-region rather than per-Jetty granularity. Table 3 projects the byte cost of closing these (per-Jetty state grows from 20 B to 48 B; the $(1024, 1024)$ ratio shifts from 4,855× to 3,855×). Two capabilities present in Ascend silicon are out-of-scope: a hardware collective engine on top of URMA, and a compact transport mode with reliability delegated to the lower layer. OpenURMA exposes the underlying verbs and implements the full reliable transport; the unordered service mode already behaves close to compact, but exposing it as a distinct opcode set is follow-on.

Comparison framing. OpenRoCE is an apples-to-apples baseline, not an optimised production stack; it anchors the comparison on identical infrastructure (same toolchain, target, harness). We do not address "how does OpenURMA compare to ConnectX-7?" — that would require disentangling the protocol design from a vendor's many-product-cycle optimisation budget. The clean-slate competitors discussed in §11 are likewise unavailable as in-fabric baselines: SRNIC, StaR, Aquila, Falcon, and Ascend itself are vendor-internal silicon; UEC is a spec without public RTL; the IRN-derived loss-recovery line is algorithm-and-simulation. None reduces to synthesisable RTL without effectively re-implementing it. We treat them as architectural reference points (Table 4) and use OpenRoCE as the single in-fabric baseline because it is the only design point where every variable below the protocol can be held fixed.

When UB does *not* win. The headline 4.37× is on the worst point for RoCE: a 64 B cold-miss READ where the entire PCIe round-trip is on the critical path. The gap narrows in three regimes already documented: (i) bulk transfers in the bandwidth-bound regime (§7.9), where all stacks converge within ∼5 % at 16–64 KB; (ii) RoCE BF inline-WQE workloads on the WRITE/SEND side, where one of the four PCIe traversals elides and the ratio falls below 2×; (iii) small clusters ($N < \sqrt{512} \approx 23$, §7.10) where RoCE's QP cache does not spill and the per-op latency floor is the only gap. UB also still pays its own pipeline overhead — the URMA path is 24 cycles cold-start vs RoCE's 9 — which is a real 46 ns disadvantage on the work-queue path that the on-bus controller and PCIe elision compensate for, but do not erase.

Security and isolation. OpenURMA enforces the access-control token at memory-region granularity, not per-Jetty as the spec permits. Tightening this is straightforward (a token field in the per-Jetty record extends per-Jetty state by 4 B; the per-op check becomes a table read on the already-fetched Jetty descriptor) but was deferred to keep the MVP surface tight. The bounded-state property removes one of the cache-pressure sources prior tenant-isolation work [22, 32] exploits, so the multi-tenant story is structurally cleaner under UB than under per-QP RoCE; we do not claim full isolation.

Power, DSPs, and the off-FPGA path. Out-of-context Vivado does not report dynamic power, and our LUT- and BRAM-only accounting is a poor proxy for ASIC die area. The DSP count of 3 (Table 2-adjacent figures) reflects that the transport and transaction layers are control-heavy and arithmetic-light — the only DSPs we use are in the exponential-backoff RTO timer's shift-multiply. On a production tape-out, the missing surfaces (the platform shell, the host-bus interface, the wider-flit crossbars we noted as HLS-instantiation artifacts) are what would set the power budget; OpenURMA's contribution is the transport/transaction layer specifically, not the surrounding glue.

11. Related Work

§2.2 grouped prior work on RDMA scale into three buckets — software above the device, hardware inside the device, and programmable substrates — and observed that none of the three removes both costs of the peripheral-NIC abstraction. We use the same lens to organise this section, then separately survey clean-slate competitor transports that, like UB, propose new abstractions rather than patch the old one. Table 4 summarises the landscape.

Software above the device. The earliest sustained attempts at RDMA scale worked entirely above the verb interface. FaSST [18] dropped to unreliable datagrams and reimplemented reliability in software, shrinking per-QP state at the cost of moving the reliability contract into the application. 1RMA [41] removed per-connection state entirely by issuing one-sided RDMAs over UDP with per-operation authentication, but the resulting protocol surface is read/write only and loses RC's per-connection ordering. Pony Express and the broader SmartNIC-pacing line take the same stance: the device is given, the work-queue / completion protocol is given, only what runs above can change. These approaches reduce one symptom (per-QP state) but leave the peripheral attachment and its four PCIe traversals in place.

Hardware inside the device. A long line of work has changed the NIC's internals while preserving the Queue-Pair-over-PCIe envelope. The connection- state direction

Table 4: Design-point positioning vs. recent RDMA scalability work (2018–2025). Rows are grouped by primary contribution. "Conn. state" is what the NIC stores per peer pair / per connection; "Loss recov." is the on-NIC loss-recovery scheme; "Multi-path" is whether the transport admits multi-path delivery without reorder breakage; "Ordering surface" is what semantic ordering the spec exposes; "Substrate" is the realisation. UB/OpenURMA is the only point that combines per-host-pair connection state, a four-axis opt-in ordering surface, and an open FPGA-RTL substrate.

Work	Conn. state	Loss recov.	Multi-path	Ordering surface	Substrate
RoCEv2 RC	per-QP	GBN	–	strict / QP	vendor ASIC
FaSST [18]	UD (per msg)	app-level	–	none	SW + commodity RNIC
1RMA [41]	none	per-op auth	yes	none	SW + RoCE
Aquila + 1RMA [9]	cell, none	cell-level	cell-network	none	custom ASIC
SRNIC [46]	QP cache + spill	RoCE	–	strict / QP	vendor ASIC
StaR [45]	reconstructed	RoCE	–	strict / QP	vendor ASIC
Falcon [42]	per-conn	SACK + RTT	yes	per-flow	vendor ASIC
UEC v1.0 [44]	packet-sprayed	SACK	yes	per-transaction	industry spec
Tonic [3]	programmable	programmable	programmable	programmable	FPGA (Verilog)
NanoTransport [17]	programmable	programmable	programmable	programmable	FPGA (Chisel/P4)
StRoM [40]	per-QP	RoCE-derived	–	per-QP	FPGA (HLS)
Flor [30]	per-QP	RoCE	–	per-QP	open framework
SwCC [12]	per-QP	RoCE + SW CC	–	per-QP	NIC + RISC-V
IRN [35]	per-QP	SR + per-pkt ACK	–	strict / QP	RoCE delta
MELO [33]	per-QP	const-mem SR	–	strict / QP	algorithm + sim
FaSR [13]	per-QP	line-rate SR	–	strict / QP	RNIC
DCP (SIGCOMM'25) [31]	per-QP	RTO-free SR	yes (pkt-LB)	strict / QP	hardware
LEFT [14]	per-QP	shared bitmap	yes	strict / QP	RNIC + sim
MP-RDMA [34]	per-QP + OOO	RoCE	yes	OOO + bitmap	RoCE delta
ConWeave [43]	per-QP	RoCE	yes	per-QP	switch + NIC
STrack [24]	per-flow	SR + fast recov	yes	per-flow	hardware
Spectrum-X [37]	per-QP	RoCE	yes	per-QP	NIC + switch
Justitia [47]	per-QP	–	–	–	SW userspace
Husky [22]	per-QP (diag)	–	–	–	test suite
Harmonic [32]	per-QP	–	–	–	hardware (BF-3)
SCR [48]	SW-programmable	SW-prog	SW-prog	SW-prog	BlueField-3 + DPA
UB / OpenURMA	**per host-pair**	**GBN + TPSACK**	**TPG + spray**	**four-axis opt-in**	**open FPGA RTL**

— SRNIC [46] (on-chip cache with host-DRAM spill), StaR [45] (per-connection state reconstructed on demand from packet metadata) — reduces the on-chip footprint without changing what the QP fundamentally is. Meta's production RoCE deployment [8] reports needing up to 32 QPs per source–destination pair to approach roofline throughput, an empirical face of the same QP-binds-paths problem. The loss-recovery direction — IRN [35], MELO [33], FaSR [13], DCP [31], LEFT [14] — replaces Go-Back-N with bitmap-tracked selective retransmission and bounds the bitmap state. The multi-path direction — MP-RDMA [34], ConWeave [43], STrack [24], Spectrum-X [37] — admits packet spreading with bitmap-tracked per-transaction completion. These are real improvements within the abstraction, and UB inherits algorithmic ideas from them: its selective-acknowledgement path is the spec-level descendant of IRN, and the multi-path spreading reuses MP-RDMA's relaxed- ordering observation. The difference is structural: in UB the multi-path observation becomes an architectural default of the unordered service mode and is correctness-safe through the two-reorder-buffer split (§3.2) rather than through per-transaction bitmap state on the wire.

Programmable substrates. A third line builds FPGA or programmable-hardware fabrics that *host* arbitrary transports rather than proposing one. Tonic [3] (Verilog), NanoTransport [17] (P4/Chisel), StRoM [40] (HLS), Flor [30] (open heterogeneous-RNIC framework), SwCC [12] (on-NIC RISC-V for software-programmable congestion control), NICA [6], AccelNet [7], and ClickNP [27] (with KV-Direct [26] as an early ClickNP-style RDMA offload) are substrates on which new transports can be expressed. SCR [48] pushes the substrate idea into commercial silicon by exposing packet-granular software control on top of a vendor hardware transport. OpenURMA builds on OpenClickNP [25] as its substrate, but the contribution is not a new substrate; it is a new protocol expressed in synthesisable RTL on it, with per-element area, BRAM, and post-route timing numbers the architecture's costs can be argued from.

Clean-slate transport competitors. Three other proposals leave the QP-over-PCIe abstraction behind, with sharply different choices on what they replace it with. Aquila [9] pairs a custom intra-rack fabric with 1RMA cells: it removes per-host-pair state but its scope is a single clique and its design is fabric-specific. *Aquila bus-ified one rack; UB bus-ifies an arbitrary host that has the controller on-bus.* Google's Falcon [42] adds hardware-SACK loss recovery, RTT-based shaping, and PSP-encrypted multi-path but retains a per-connection abstraction in which connection state still fuses identity with transport. *Falcon optimises inside the per-pair envelope; UB removes the envelope.* Ultra Ethernet 1.0 [44] defines packet-sprayed, SACK-based reliable transport with per-transaction ordering guarantees implemented via per-transaction bitmap tracking on the wire. *UEC pays for multi-path safety with per-transaction SACK state; UB gets the same safety for free by separating its two*

reorder buffers. The only shipping silicon for UB itself is Ascend 950 [16]; the silicon is closed and publishes no per-verb latency or per-element area, so the architectural choices are derivable from the spec but the implementation cost is not. OpenURMA fills that gap.

Memory-semantic fabrics. CXL [5] is a load/store fabric over PCIe physical layers; NVLink [36] is a vendor-proprietary intra-server fabric. The standard framing treats CXL as a niche distinct from scale-out RDMA, but a sharper framing is that the two are different layers of the same architectural axis: CXL provides memory-semantic access at intra-chassis distances; UB's load/store path provides memory-semantic access at cross-chassis distances. They compose. A future system can route the same ISA load instruction to CXL inside the chassis and to UB across the rack without an API change. The thing that permits the composition is UB's removal of the PCIe boundary on remote access; CXL by itself terminates at the chassis.

Multi-tenant isolation, congestion control, total order. Three further dimensions are tangential to the peripheral-NIC argument but worth noting briefly. Justitia [47], Husky [22], and Harmonic [32] attack RNIC tenant isolation in software and hardware respectively; the layer split removes one of the cache-pressure sources Husky exploits (the connection cache no longer scales with application-pair count). DCQCN [49] is RoCE's de-facto congestion control and serves as the OpenRoCE baseline's CC; UB defines its own queue-occupancy-based controller whose dynamics we measure in §8.7. At the opposite extreme from UB's opt-in ordering, 1Pipe [29] provides causally and totally ordered datacenter-wide communication at the cost of in-network barrier aggregation and an extra RTT per message; that contract is right for a higher-level transaction layer that may sit above URMA but wrong for the verb path itself, whose workloads are bandwidth-bound and per-pair-scoped.

12. Conclusion

OpenURMA is the first clean-room open implementation of the Unified Bus protocol's transport and transaction layers. The artifact realises the three architectural commitments the spec makes — a layer split that bounds per-NIC state additively in N and M, an ordering surface that applications opt into rather than always pay, and a load/store data path on the on-chip bus — in 39 synthesisable elements that close 322 MHz post-route on commodity FPGA, with a matched OpenRoCE baseline on the same toolchain, the same target part, and the same test harness.

What the implementation measures. Bounded state is asymptotic, not constant: $5\times$ less state than RoCE RC at $(N, M)=(1, 1)$, $4{,}855\times$ less at $(1024, 1024)$, crossing the on-chip-SRAM-to-host-DRAM spill boundary between $N{\approx}23$ and $N{\approx}1024$. Opt-in ordering costs 13 KLUT (10.6% of the design's LUT budget) and *zero* pipeline cycles on operations that do not request it. The load/store path emits its first wire flit in 8 cycles ($\approx$25 ns at 322 MHz); end-to-end CPU-to-remote-DRAM-to-CPU round trip on a 64-byte READ measures $\approx$500 ns, $4.37\times$ below the matched RoCE-DMA baseline (2186 ns). The gap is structural: the nine PCIe-bound cost components on the RoCE round trip are protocol consequences of disjoint CPU/NIC address spaces; the load/store path removes the disjunction. The whole design fits in $\sim$14% of an Alveo U50's LUT budget.

Why the open artifact matters. The architecture is Huawei's; Ascend 950 ships it in commodity silicon. What did not exist until now is a way for the community to measure the architecture, compare its operating points against RoCE on identical infrastructure, and prototype follow-on transports against UB-style assumptions. The implementation also makes visible a coupling the spec asserts but cannot prove on paper: each of the three architectural moves depends on the others; removing any one and keeping the others on a PCIe-attached NIC reproduces RoCE's costs. We release the artifact as a substrate for follow-on work — multi-flit retransmission, alternative congestion control, comparison against Falcon, UEC, and SRNIC/StaR-derivative transports on the gem5 scaffold — and as a controlled reference any such comparison can re-use.

Code. `https://github.com/bojieli/OpenURMA`.

Acknowledgments. This work builds on the OpenClickNP toolchain (an open re-implementation of the ClickNP element model) and reads the UB-Base-Specification 2.0.1 as authoritative; implementation choices, modeling assumptions, and the measurement framework are ours, and any deviations from the spec are unintentional.

References

[1] Hasan Al Maruf and Mosharaf Chowdhury. Effectively prefetching remote memory with Leap. In *Proc. USENIX ATC*, 2020. Far-memory prefetch heuristic; cited in §7.8 as an example of software-side swap optimisation.

[2] Emmanuel Amaro, Christopher Branner-Augmon, Zhihong Luo, Amy Ousterhout, Marcos K. Aguilera, Aurojit Panda, Sylvia Ratnasamy, and Scott Shenker. Can far memory improve job throughput? In *Proc. EuroSys*, 2020. Introduces Fastswap; reports $\sim$1 µs kernel-side overhead and batched-prefetch swap-in, the basis of the second swap profile in §7.8.

[3] Mina Tahmasbi Arashloo, Alexey Lavrov, Manya Ghobadi, Jennifer Rexford, David Walker, and David Wentzlaff. Enabling programmable transport

protocols in high-speed NICs. In *Proc. USENIX NSDI*, 2020.

[4] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. Benchmarking cloud serving systems with YCSB. In *Proc. ACM SoCC*, 2010. The Yahoo! Cloud Serving Benchmark; we use the YCSB-A 50/50 Get-Put Zipfian workload in §8.8.

[5] CXL Consortium. Compute Express Link (CXL) Specification 3.1. `https://www.computeexpresslink.org/`, 2024.

[6] Haggai Eran, Lior Zeno, Maroun Tork, Gabi Malka, and Mark Silberstein. NICA: An infrastructure for inline acceleration of network applications. In *Proc. USENIX ATC*, 2019.

[7] Daniel Firestone, Andrew Putnam, Sambhrama Mundkur, Derek Chiou, Alireza Dabagh, Mike Andrewartha, Hari Angepat, et al. Azure Accelerated Networking: SmartNICs in the public cloud. In *Proc. USENIX NSDI*, 2018.

[8] Adithya Gangidi, Rui Miao, Shengbao Zheng, Sai Jayesh Bondu, Guilherme Goes, Hany Morsy, Rohit Puri, Mohammad Riftadi, Ashmitha Jeevaraj Shetty, Jingyi Yang, Shuqiang Zhang, Mikel Jimenez Fernandez, Shashidhar Gandham, and Hongyi Zeng. RDMA over Ethernet for distributed training at meta scale. In *Proc. ACM SIGCOMM*, 2024.

[9] Dan Gibson, Hema Hariharan, Eric Lance, Moray McLaren, Behnam Montazeri, Arjun Singh, Stephen Wang, Hassan M. G. Wassel, Zhehua Wu, Sunghwan Yoo, Raghuraman Balasubramanian, Prashant Chandra, Michael Cutforth, Peter Cuy, David Decotigny, Rakesh Gautam, Alex Iriza, Milo M. K. Martin, Rick Roy, Zuowei Shen, Ming Tan, Ye Tang, Monica Wong-Chan, Joe Zbiciak, and Amin Vahdat. Aquila: A unified, low-latency fabric for datacenter networks. In *Proc. USENIX NSDI*, 2022.

[10] Juncheng Gu, Youngmoon Lee, Yiwen Zhang, Mosharaf Chowdhury, and Kang G. Shin. Efficient memory disaggregation with Infiniswap. In *Proc. USENIX NSDI*, 2017. Kernel-side overhead of 3–5 µs on the swap-in path is the parameter referenced in §7.8.

[11] Tingbo He. A time scaling theory for multilayer electronic systems. *ChinaXiv*, May 2026. chinarxiv-202605.00224. Perspective from Huawei Semiconductor: τ scaling as successor to geometric Moore's-Law scaling; positions Unified Bus as the system-layer τ reduction mechanism with end-to-end remote-access latency from $\sim$10s of μs (TCP/IP-class) to $\sim$100 ns.

[12] Hongjing Huang, Jie Zhang, Xuzheng Chen, Ziyu Song, Jiajun Qin, and Zeke Wang. SwCC: Software-programmable and per-packet congestion control in RDMA engine. In *Proc. USENIX ATC*, 2025.

[13] Peihao Huang, Guo Chen, Xin Zhang, Can Liu, Hongyu Wang, Huijun Shen, Ying Bian, Yuanwei Lu, Zhenyuan Ruan, Bojie Li, Jiansong Zhang, Yongfeng Liu, and Zhigang Chen. Fast and scalable selective retransmission for RDMA. In *Proc. IEEE INFOCOM*, 2025.

[14] Peihao Huang, Xin Zhang, Zhigang Chen, Can Liu, and Guo Chen. LEFT: Lightweight and fast packet reordering for RDMA. In *Proc. APNet*, 2024.

[15] Huawei Technologies. UB-base-specification 2.0.1. `https://www.unifiedbus.org/`, 2024. Unified Bus consortium specification, available from the consortium's documentation portal.

[16] Huawei Technologies. Ascend 950 NPU architecture white paper. Huawei vendor white paper, May 2026. Architectural disclosure for the Ascend 950PR and 950DT NPUs; first publicly documented silicon implementing the Unified Bus spec, with URMA (asynchronous Write/Read/Send/Atomic via Jetty) and UB Memory (synchronous Load/Store + AtomicStore/Load/Swap/CAS) exposed as distinct on-die paths, plus URMA-CTP / URMA-TP transport modes, UBoE 2$\times$400 Gbps, a hardware Collective Communication Unit (CCU), UB On-Chip Switch for in-die forwarding, and a UB super-node target of 8192 cards (cluster target $>$128K).

[17] Stephen Ibanez, Alex Mallery, Serhat Arslan, Theo Jepsen, Muhammad Shahbaz, Nick McKeown, and Changhoon Kim. NanoTransport: A low-latency, programmable transport layer for NICs. In *Proc. ACM SOSR*, 2021.

[18] Anuj Kalia, Michael Kaminsky, and David G. Andersen. Using RDMA efficiently for key-value services. In *Proc. ACM SIGCOMM*, 2014.

[19] Anuj Kalia, Michael Kaminsky, and David G. Andersen. FaSST: Fast, scalable and simple distributed transactions with two-sided (RDMA) datagram RPCs. In *Proc. USENIX OSDI*, 2016.

[20] Anuj Kalia, Michael Kaminsky, and David G. Andersen. Datacenter RPCs can be general and fast. In *Proc. USENIX NSDI*, 2019.

[21] Antoine Kaufmann, Tim Stamler, Simon Peter, Naveen Kr. Sharma, Arvind Krishnamurthy, and Thomas Anderson. TAS: TCP acceleration as an OS service. In *Proc. EuroSys*, 2019. Reports detailed PCIe-class transaction latency decompositions used as parameter references in §7.

[22] Xinhao Kong, Jingrong Chen, Wei Bai, Yechen Xu, Mahmoud Elhaddad, Shachar Raindel, Jitendra Padhye, Alvin R. Lebeck, and Danyang Zhuo.

Understanding RDMA microarchitecture resources for performance isolation. In *Proc. USENIX NSDI*, 2023.

[23] Xinhao Kong, Yibo Zhu, Huaping Zhou, Zhuo Jiang, Jianxi Ye, Chuanxiong Guo, and Danyang Zhuo. Collie: Finding performance anomalies in RDMA subsystems. In *Proc. USENIX NSDI*, 2022.

[24] Yanfang Le, Rong Pan, Peter Newman, Jeremias Blendin, Abdul Kabbani, Vipin Jain, Raghava Sivaramu, and Francis Matus. STrack: A reliable multipath transport for AI/ML clusters. arXiv:2407.15266, 2024.

[25] Bojie Li. OpenClickNP: a clean-room reimplementation of ClickNP on Alveo U50. `https://github.com/bojieli/OpenClickNP`, 2025–2026.

[26] Bojie Li, Zhenyuan Ruan, Wencong Xiao, Yuanwei Lu, Yongqiang Xiong, Andrew Putnam, Enhong Chen, and Lintao Zhang. KV-Direct: High-performance in-memory key-value store with programmable NIC. In *Proc. ACM SOSP*, 2017.

[27] Bojie Li, Kun Tan, Layong Larry Luo, Yanqing Peng, Renqian Luo, Ningyi Xu, Yongqiang Xiong, Peng Cheng, and Enhong Chen. ClickNP: Highly flexible and high-performance network processing with reconfigurable hardware. In *Proc. ACM SIGCOMM*, 2016.

[28] Bojie Li, Zhilong Xiang, Xiang Wang, Hongru Jonathan Zhou, and Kun Tan. FastWake: Revisiting host network stack for interrupt-mode RDMA. In *Proc. APNet*, 2023.

[29] Bojie Li, Gefei Zuo, Wei Bai, and Lintao Zhang. 1Pipe: Scalable total order communication in data center networks. In *Proc. ACM SIGCOMM*, 2021.

[30] Qiang Li, Yixiao Gao, Xiaoliang Wang, Haonan Qiu, Yanfang Le, Derui Liu, Qiao Xiang, Fei Feng, Peng Zhang, Bo Li, Jianbo Dong, Lingbo Tang, Hongqiang Harry Liu, Shaozong Liu, Weijie Li, Rui Miao, Yaohui Wu, Zhiwu Wu, Chao Han, Lei Yan, Zheng Cao, Zhongjie Wu, Chen Tian, Guihai Chen, Dennis Cai, Jinbo Wu, Jiaji Zhu, Jiesheng Wu, and Jiwu Shu. Flor: An open high performance RDMA framework over heterogeneous RNICs. In *Proc. USENIX OSDI*, 2023.

[31] Wenxue Li, Xiangzhou Liu, Yunxuan Zhang, Zihao Wang, Wei Gu, Tao Qian, Gaoxiong Zeng, Shoushou Ren, Xinyang Huang, Zhenghang Ren, Bowen Liu, Junxue Zhang, Kai Chen, and Bingyang Liu. Revisiting RDMA reliability for lossy fabrics. In *Proc. ACM SIGCOMM*, 2025. Best Student Paper, Honorable Mention.

[32] Jiaqi Lou, Xinhao Kong, Jinghan Huang, Wei Bai, Nam Sung Kim, and Danyang Zhuo. Harmonic: Hardware-assisted RDMA performance isolation for public clouds. In *Proc. USENIX NSDI*, 2024.

[33] Yuanwei Lu, Guo Chen, Bojie Li, Kun Tan, Yongqiang Xiong, Peng Cheng, Jiansong Zhang, Enhong Chen, and Thomas Moscibroda. Memory efficient loss recovery for hardware-based transport in datacenter. In *Proc. APNet*, 2017.

[34] Yuanwei Lu, Guo Chen, Bojie Li, Kun Tan, Yongqiang Xiong, Peng Cheng, Jiansong Zhang, Enhong Chen, and Thomas Moscibroda. Multi-Path transport for RDMA in datacenters. In *Proc. USENIX NSDI*, 2018.

[35] Radhika Mittal, Alexander Shpiner, Aurojit Panda, Eitan Zahavi, Arvind Krishnamurthy, Sylvia Ratnasamy, and Scott Shenker. Revisiting network support for RDMA. In *Proc. ACM SIGCOMM*, 2018.

[36] NVIDIA Corporation. NVLink: A high-bandwidth inter-GPU interconnect. Vendor whitepaper, 2014–2025. Successive generations of the NVLink fabric are described in the NVIDIA whitepaper series.

[37] NVIDIA Networking. NVIDIA Spectrum-X: Adaptive routing and telemetry-based congestion control for AI networks. NVIDIA technical brief, 2024. Vendor description of multi-path adaptive-routing delivery over Spectrum-4 / BlueField-3 NICs; the closest commercially-deployed point of comparison to UB's TPG multi-path scheme.

[38] Yifan Qiao, Chenxi Wang, Zhenyuan Ruan, Adam Belay, Qingda Lu, Yiying Zhang, Miryung Kim, and Guoqing Harry Xu. Hermit: Low-latency, high-throughput, and transparent remote memory via feedback-directed asynchrony. In *Proc. USENIX NSDI*, 2023. Asynchronous remote-memory swap with feedback-directed I/O; cited in §7.8 for the same workload regime as Infiniswap/Fastswap.

[39] Sebastian Ramos and Torsten Hoefler. Designing high-performance, low-latency multi-cluster communication on modern InfiniBand networks. In *Proc. ACM HPDC*, 2023. Reports ConnectX-7 PCIe round-trip latencies in the $\sim$300–500 ns range; we use this as the parameterised PCIe RTT in §7.

[40] David Sidler, Zeke Wang, Monica Chiosa, Amit Kulkarni, and Gustavo Alonso. StRoM: Smart remote memory. In *Proc. EuroSys*, 2020.

[41] Arjun Singhvi, Aditya Akella, Dan Gibson, Thomas F. Wenisch, Monica Wong-Chan, Sean Clark, Milo M. K. Martin, Moray McLaren, Prashant Chandra, Rob Cauble, et al. 1RMA: Re-envisioning remote memory access for multi-tenant datacenters. In *Proc. ACM SIGCOMM*, 2020.

[42] Arjun Singhvi, Nandita Dukkipati, Prashant Chandra, Hassan M. G. Wassel, Naveen Kr. Sharma, Anthony Rebello, Henry Schuh, Praveen Kumar, Behnam Montazeri, Neelesh Bansod, Sarin Thomas, Inho Cho, Hyojeong Lee Seibert, Baijun Wu, Rui Yang, Yuliang Li, Kai Huang, Qianwen Yin, Abhishek Agarwal, Srinivas Vaduvatha, Weihuang

Wang, Masoud Moshref, Tao Ji, David Wetherall, and Amin Vahdat. Falcon: A reliable, low latency hardware transport. In *Proc. ACM SIGCOMM*, 2025. Specification contributed to OCP 2024.

[43] Cha Hwan Song, Xin Zhe Khooi, Raj Joshi, Inho Choi, Jialin Li, and Mun Choon Chan. Network load balancing with in-network reordering support for RDMA. In *Proc. ACM SIGCOMM*, 2023.

[44] Ultra Ethernet Consortium. Ultra Ethernet specification 1.0. Industry specification, 2025. Released June 2025 under Linux Foundation JDF; `https://ultraethernet.org/`.

[45] Xizheng Wang, Guo Chen, Xijin Yin, Huichen Dai, Bojie Li, Binzhang Fu, and Kun Tan. StaR: Breaking the scalability limit for RDMA. In *Proc. IEEE ICNP*, 2021.

[46] Zilong Wang, Layong Luo, Qingsong Ning, Chaoliang Zeng, Wenxue Li, Xinchen Wan, Peng Xie, Tao Feng, Ke Cheng, Xiongfei Geng, Tianhao Wang, Weicheng Ling, Kejia Huo, Pingbo An, Kui Ji, Shideng Zhang, Bin Xu, Ruiqing Feng, Tao Ding, Kai Chen, and Chuanxiong Guo. SRNIC: A scalable architecture for RDMA NICs. In *Proc. USENIX NSDI*, 2023.

[47] Yiwen Zhang, Yue Tan, Brent Stephens, and Mosharaf Chowdhury. Justitia: Software multitenancy in hardware kernel-bypass networks. In *Proc. USENIX NSDI*, 2022.

[48] Chenxingyu Zhao, Jaehong Min, Ming Liu, and Arvind Krishnamurthy. White-boxing RDMA with packet-granular software control. In *Proc. USENIX NSDI*, 2025.

[49] Yibo Zhu, Haggai Eran, Daniel Firestone, Chuanxiong Guo, Marina Lipshteyn, Yehonatan Liron, Jitendra Padhye, Shachar Raindel, Mohamad Haj Yahia, and Ming Zhang. Congestion control for Large-Scale RDMA deployments. In *Proc. ACM SIGCOMM*, 2015.